

ВІННИЦЬКИЙ ТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ
(повне найменування закладу освіти)

Комп'ютерне відділення
(повне найменування відділення)

Випускова предметна циклова комісія комп'ютерної інженерії
(повна назва випускової предметної циклової комісії)

ДОПУСТИТИ ДО ЗАХИСТУ
Голова випускової предметної
циклової комісії
комп'ютерної інженерії
_____ Олексій ПОХИЛЬКО
_____ 2026 р.

Платформа для створення та проведення тестувань

Пояснювальна записка до кваліфікаційної роботи
за освітньо-професійним ступенем фаховий молодший бакалавр

Виконав: студент 4 – го курсу, групи 4КІ-3
галузі знань 12 Інформаційні технології
спеціальності 123 Комп'ютерна інженерія
Володимир РАТИНСЬКИЙ
(ім'я, прізвище)

Керівник Михайло КРЕНЦІН
(ім'я, прізвище)

Рецензент Анатолій КОЛЕСНИК
(ім'я, прізвище)

Вінниця - 2026 р.

ВІННИЦЬКИЙ ТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ

Комп'ютерне відділення

Випускова предметна циклова комісія комп'ютерної інженерії

Освітньо-професійний ступінь фаховий молодший бакалавр

Галузь знань 12 Інформаційні технології

Спеціальність 123 Комп'ютерна інженерія

Освітньо-професійна програма Комп'ютерна інженерія

ЗАТВЕРДЖУЮ

Голова випускової предметної
циклової комісії

комп'ютерної інженерії

_____ Олексій ПОХИЛЬКО
_____ 2026 р.

З А В Д А Н Н Я ДО КВАЛІФІКАЦІЙНОЇ РОБОТИ СТУДЕНТУ

Ратинському Володимиру Вячеславовичу

(прізвище, ім'я, по батькові)

1. Платформа для створення та проведення тестувань

Керівник кваліфікаційної роботи Кренцін Михайло Дмитрович

(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджене наказом по коледжу від 26 березня 2026 року № 65-у

2. Строк подання студентом 02.06.2026 р.

3. Вхідні дані до проекту (роботи): Програмні засоби розробки: IntelliJ IDEA Community Edition та Visual Studio Code. Для реалізації системи використовуються мови програмування Java та JavaScript, фреймворки Spring Boot і React, система керування базами даних MySQL, а також веббраузер із підтримкою HTML5, CSS3 та JavaScript.

4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити): Вступ; 1 Техніко-економічне обґрунтування технічного завдання; 2 Теоретична частина; 3 Проєктна частина; 4 Економічна частина; 5 Охорона праці; Висновки; Список використаних джерел; Додатки.

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень): Діаграма варіантів використання(Use Case Diagram); Діаграма класів(Class Diagram); ER-діаграма бази даних(ER-Diagram); Діаграма послідовності основних сценарії роботи системи(Sequence Diagram); Інтерфейси користувача; Схема взаємодії клієнтської та серверної частини.

6. Консультанти розділів кваліфікаційної роботи

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		Завдання видав	Завдання прийняв
Економічна частина	Олександра АЛЕКСЄЄНКО		
Охорона праці	Тетяна ПОГОСЯН		
Нормоконтроль	Тетяна ШИДЛОВСЬКА		

7. Дата видачі завдання 01.04.2026

КАЛЕНДАРНИЙ ПЛАН

Назва етапів кваліфікаційної роботи	Строк виконання етапів роботи	Примітка
Аналіз технічного завдання	до 12.04	
Вступ, техніко-економічне обґрунтування технічного завдання	до 19.04	
Вибір технології та розробка проекту	до 01.05	
Проектна частина	до 12.05	
Тестування проєкту	до 18.05	
Теоретична частина	до 04.05	
Оформлення пояснювальної записки	до 28.05	
Охорона праці	до 30.05	
Економічні розрахунки	до 02.06	

Студент

_____ (підпис)

Володимир РАТИНСЬКИЙ

(ім'я, прізвище)

Керівник
кваліфікаційної роботи

_____ (підпис)

Михайло КРЕНЦІН

(ім'я, прізвище)

Анотація

Кваліфікаційна робота присвячена розробці вебплатформи для створення та проведення тестування. Основною метою проєкту є створення сучасного програмного засобу, який дозволяє викладачам створювати тестові завдання та контролювати результати тестування, а студентам – проходити тести, отримувати оцінки та відстежувати власний прогрес навчання.

Платформа розроблена з використанням інтегрованих середовищ розробки IntelliJ IDEA Community Edition та Visual Studio Code. Для реалізації клієнтської частини застосовано бібліотеку React, яка забезпечує створення інтерактивного користувацького інтерфейсу та зручею взаємодію користувача із системою. Серверна частина реалізована за допомогою фреймворку Spring Boot мовою програмування Java. Для зберігання та обробки даних використовується система керування базами даних MySQL.

Функціональні можливості платформи включають реєстрацію та авторизацію користувачів, розподіл ролей(адміністратор, викладач, студент), створення та редагування тестів, підтримку різних типів питань, завантаження графічних матеріалів до тестових завдань, проведення тестування, автоматичне оцінювання результатів та перегляд статистики успішності. Також реалізовано механізм створення приватних тестів із кодом доступу та обмеженим терміном дії.

Використання технологій JavaScript(React) та Java(Spring Boot) забезпечує високу продуктивність, масштабованість і зручність подальшого розвитку програмного продукту. База даних MySQL забезпечує надійне зберігання інформації про користувачів, тести, питання, відповіді та результати тестування.

Результатом виконання кваліфікаційної роботи є функціональна вебплатформа для створення та проведення тестування, яка може бути використанна в освітніх закладах для організації контролю знань.

Annotation

The qualification work is devoted to the development of a web platform for creating and conducting tests. The main objective of the project is to create a modern software solution that enables teachers to create test tasks and monitor testing results, while allowing students to take tests, receive grades, and track their learning progress.

The platform was developed using the IntelliJ IDEA Community Edition and Visual Studio Code integrated development environments. The client-side application was implemented using the React library, which provides an interactive user interface and convenient interaction with the system. The server-side application was implemented using the Spring Boot framework and the Java programming language. MySQL was used as the database management system for storing and processing data.

The platform's functionality includes user registration and authentication, role-based access control (administrator, teacher, student), creation and editing of tests, support for various question types, uploading assessment of results, and viewing performance statistics. A mechanism for creating private tests with access codes and limited validity periods was also implemented.

The use of JavaScript (React) and Java (Spring Boot) technologies ensures high performance, scalability, and ease of further development of the software product. The MySQL database provides reliable storage of information about users, tests, questions, answers, and test results.

The result of the qualification work is a functional web platform for creating and conducting tests, which can be used in educational institutions to organize knowledge assessment.

Зміст

Вступ.....	8
1 Техніко-економічне обґрунтування технічного завдання	10
1.1 Постановка задачі	10
1.2 Вибір технологій.....	11
1.3 Економічне обґрунтування.....	13
1.4 Очікувані результати.....	14
2 Теоретична частина.....	16
2.1 Веборієнтовані системи тестування	16
2.2 Клієнтські вебтехнології та бібліотека React.....	17
2.3 Серверна розробка з використанням Spring Boot	18
2.4 Система керування базами даних MySQL	19
2.5 Автентифікація та авторизація користувачів	20
2.6 Висновки до теоретичної частини	21
3 Проектна частина	22
3.1 Проектування архітектури вебплатформи	22
3.2 Проектування бази даних	24
3.3 Реалізація серверної частини.....	26
3.4 Реалізація клієнтської частини.....	29
3.5 Реалізація системи ролей та доступу.....	37
3.6 Реалізація тестування та обробки результатів.....	39
3.7 Реалізація приватних тестів і завантаження зображень	42
3.8 Висновки до проектної частини.....	43
4 Економічна частина	44
4.1 Характеристика роботи та її призначення	44

					КР ФМБ 2026 123.3 013 ПЗ							
Змн.	Лист	№ докум.	Підпис	Дата								
Розроб.		Ратинський В.В.			Платформа для створення та проведення тестування Пояснювальна записка		Лім.		Арк.		Аркушів	
Перевір.		Кренцін М.Д.							6	64		
Реценз.		Колесник А.І.		Гр. 4КІЗ								
Н. Контр.		Шидловська Т.І.										
Затверд.		Щегоцька Н.М.										

4.2	Розрахунок трудомісткості створення вебплатформи.....	44
4.3	Розрахунок витрат на оплату праці розробника.....	45
4.4	Амортизаційні відрахування та витрати на електроенергію	47
4.5	Розрахунок собівартості програмного продукту.....	49
5	Охорона праці.....	52
5.1	Оцінка умов роботи під час розробки програмного продукту	52
5.2	Організація безпечного та ергономічного робочого місця	53
5.3	Санітарно-гігієнічні умови праці користувача ПК	54
5.4	Вимоги електробезпеки під час роботи з комп'ютерною технікою ...	56
5.5	Пожежна безпека робочого приміщення	57
5.6	Профілактика перевантажень під час роботи за комп'ютером	58
	Висновки	59
	Список використаних джерел	60
	Додатки.....	Error! Bookmark not defined.

Вступ

Сучасний розвиток інформаційних технологій значно впливає на організацію освітнього процесу. Навчальні заклади все частіше використовують цифрові засоби для проведення занять, перевірки знань, збереження результатів і взаємодії між викладачами та студентами. Особливо актуальним це стало в умовах поширення дистанційного та змішаного навчання, коли виникає потреба у зручних веборієнтованих системах, доступних із будь-якого пристрою через браузер.

Актуальність теми полягає в тому, що процес створення, проведення та перевірки тестувань потребує автоматизації. Традиційна перевірка знань займає багато часу, залежить від людського фактора та не завжди дозволяє швидко отримати статистику успішності студентів. Використання вебплатформи для тестування дає змогу викладачам швидше створювати тестові завдання, автоматично перевіряти відповіді, переглядати результати студентів і зберігати дані в єдиній системі.

Крім того, для українських навчальних закладів важливо мати власні зручні програмні рішення для організації тестування. Використання сторонніх сервісів не завжди є оптимальним, оскільки вони можуть мати обмежену функціональність, недостатню локалізацію, залежність від зовнішніх платформ або обмежений контроль над збереженням даних. Тому розробка власної вебплатформи для створення та проведення тестувань є актуальним і практично важливим завданням.

Метою даної кваліфікаційної роботи є розробка вебплатформи для створення та проведення тестувань, яка забезпечує зручний інтерфейс для викладачів і студентів, а також автоматизує процес оцінювання знань. Платформа реалізована із застосуванням сучасних технологій веброзробки: бібліотеки React для створення інтерактивного користувацького інтерфейсу, фреймворку Spring Boot для реалізації серверної логіки, а також системи

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		

керування базами даних MySQL для зберігання інформації. Розробка платформи виконувалася в середовищах IntelliJ IDEA Community Edition та Visual Studio Code.

Об'єктом дослідження є процес створення та проведення комп'ютерного тестування в освітньому середовищі, який включає розробку, адміністрування та проходження тестових завдань із подальшим аналізом результатів.

Практичне значення роботи полягає у створенні функціональної вебплатформи, яка може використовуватися в навчальних закладах для проведення контролю знань, організації дистанційного тестування та моніторингу успішності студентів. Розроблена система дозволяє зменшити витрати часу на перевірку результатів, підвищити об'єктивність оцінювання та забезпечити зручний доступ до тестових завдань через вебінтерфейс.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
						9
Змн.	Арк.	№ докум.	Підпис	Дата		

1 Техніко-економічне обґрунтування технічного завдання

1.1 Постановка задачі

Метою кваліфікаційної роботи є вирішення проблеми недостатньої доступності сучасних, зручних та функціональних платформ для організації тестування в освітньому середовищі шляхом розробки вебплатформи для створення та проведення тестування. Розроблена система призначена для використання викладачами та студентами навчальних закладів і забезпечує можливість створення тестових завдань, проведення тестування, автоматичного оцінювання результатів та відстеження успішності користувачів.

Основними вимогами до проєкту є створення інтуїтивно зрозумілого користувацького інтерфейсу, реалізація надійної серверної логіки для обробки запитів, інтеграція з базою даних для зберігання інформації про користувачів, тести, питання, відповіді та результати тестування. Також важливими вимогами є забезпечення безпеки даних користувачів за допомогою сучасних механізмів автентифікації та авторизації, підтримка різних ролей користувачів, можливість створення приватних тестів із кодом доступу та підтримка графічних матеріалів у тестових завданнях.

У сучасних закладах освіти контроль знань усе частіше здійснюється за допомогою цифрових технологій. Проте багато навчальних закладів використовують сторонні онлайн-сервіси, які не завжди забезпечують необхідний рівень функціональності, безпеки даних, локалізації та можливостей адаптації під потреби конкретного закладу освіти. Наприклад, для проведення тестування широко використовуються зовнішні платформи, функціональність яких обмежена умовами використання сервісу та не дозволяє повністю контролювати процес зберігання й обробки даних.

Особливо актуальною є потреба у створенні власних веборієнтованих систем тестування для українських освітніх установ, які забезпечують

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		10

незалежність від сторонніх сервісів, підтримку української мови, можливість подальшого розширення функціоналу та адаптацію до специфічних потреб навчального процесу. Важливим фактором також є необхідність використання програмних продуктів, які не залежать від іноземених сервісів та забезпечують повний контроль над зберіганням і обробкою даних користувачів.

Запропонована платформа дозволяє автоматизувати процес створення та проведення тестування, спростити перевірку результатів, забезпечити швидкий доступ до статистики успішності та підвищити ефективність контролю знань студентів. Використання сучасних вебтехнологій забезпечує доступність системи через веббраузер без необхідності встановлення додаткового програмного забезпечення, що робить її зручною для використання як в аудиторії, так і під час дистанційного навчання.

1.2 Вибір технологій

Для розробки вебплатформи для створення та проведення тестувань необхідно було обрати технології, які забезпечують стабільну роботу системи, зручність розробки, можливість подальшого розширення функціоналу та підтримку клієнт-серверної архітектури. Під час вибору технологій було розглянуто мови програмування, фреймворки, системи керування базами даних і середовища розробки.

Для реалізації клієнтської частини можна використовувати звичайні HTML, CSS і JavaScript або сучасні бібліотеки та фреймворки. Звичайний JavaScript підходить для невеликих сторінок, однак для складної системи з різними ролями користувачів, сторінками, формами та динамічним оновленням даних його використання ускладнює підтримку коду. Angular є потужним фреймворком, але має складнішу структуру та вищий поріг входження. Vue.js є простішим у використанні, але React має ширшу

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		11

популярність, велику кількість готових бібліотек і зручний компонентний підхід [1]. Саме тому для клієнтської частини було обрано React.

Для серверної частини було розглянуто кілька можливих варіантів: Node.js, Django, Flask та Spring Boot. Node.js добре підходить для швидкої розробки серверів на JavaScript, однак для побудови структурованої системи з ролями, авторизацією та роботою з базою даних Spring Boot є більш зручним і надійним рішенням. Django та Flask використовуються з мовою Python і також дозволяють створювати вебзастосунки, проте для даного проєкту було обрано Java та Spring Boot, оскільки цей фреймворк добре підходить для створення REST API, має зручну структуру проєкту та підтримує інтеграцію з базами даних і засобами безпеки [2]. Тому для серверної частини було обрано мову Java та фреймворк Spring Boot.

Мова Java була обрана для серверної частини завдяки її надійності, строгій типізації, стабільності та широкому використанню у веброботі. Spring Boot спрощує створення серверної частини, дозволяє швидко налаштувати контролери, сервіси, репозиторії, роботу з базою даних і механізми авторизації. Це важливо для платформи тестування, оскільки сервер відповідає за реєстрацію користувачів, вхід у систему, створення тестів, додавання питань, перевірку відповідей і збереження результатів.

Для зберігання даних було розглянуто реляційні та нереляційні бази даних. Нереляційні бази даних зручні для гнучкого зберігання документів, однак у даному проєкті між основними сутностями існують чіткі зв'язки. Тому більш доцільним є використання реляційної бази даних. Серед можливих варіантів, таких як PostgreSQL і MySQL, було обрано MySQL, оскільки вона є поширеною, надійною, безкоштовною та має зрозумілу документацію для роботи з таблицями, запитам, первинними та зовнішніми ключами [3].

Для розробки серверної частини було обрано середовище розробки IntelliJ IDEA Community Edition. Це середовище добре підходить для роботи з Java та Spring Boot, має зручне автодоповнення коду, перевірку помилок,

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		12

підтримку структури проєкту та засоби для запуску й налагодження застосунку [4]. Для клієнтської частини було використано Visual Studio Code, оскільки це легке та зручне середовище для роботи з JavaScript, React, HTML, CSS і SCSS. Воно має велику кількість розширень, зручний редактор коду та інтеграцію з терміналом [5].

Для оформлення інтерфейсу користувача використовуються каскадні таблиці стилів і препроцесор SCSS. SCSS дозволяє зручніше структурувати стилі, використовувати змінні, вкладені правила та повторно застосовувати однакові елементи оформлення [6].

Для обміну даними між клієнтською та серверною частинами обрано бібліотеку Axios, яка дозволяє надсилати HTTP-запити до REST API та обробляти відповіді сервера [7].

Для переходу між сторінками вебзастосунку було використано React Router, який забезпечує маршрутизацію без повного перезавантаження сторінки [8].

Для розробки клієнтської частини також обрано допоміжні інструменти веброботи. Webpack застосовується для збирання модулів і ресурсів вебзастосунку [9]. NPM використовується для встановлення та керування JavaScript-бібліотеками, необхідними для роботи React-застосунку [10]. Node.js забезпечує середовище виконання JavaScript та використовується для запуску інструментів розробки клієнтської частини [11].

Для серверної частини застосовується Maven, який дозволяє керувати залежностями Java-проєкту, збирати застосунок і спрощувати налаштування Spring Boot-проєкту [12].

1.3 Економічне обґрунтування

Розробка вебплатформи для створення та проведення тестувань є економічно доцільною, оскільки для її реалізації використовуються сучасні

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		13

програмні засоби та технології з відкритим вихідним кодом. Це дозволяє суттєво знизити витрати на створення та впровадження програмного продукту, оскільки відсутня необхідність придбання дорогих ліцензій на програмне забезпечення.

Основними витратами під час реалізації проєкту є витрати часу на проєктування, розробку, тестування та налагодження системи. Для створення платформи використовуються безкоштовні середовища розробки IntelliJ IDEA Community Edition та Visual Studio Code, а також безкоштовні програмні технології React, Spring Boot і MySQL.

Додаткові витрати можуть бути пов'язані з розміщенням вебплатформи на сервері та її подальшою технічною підтримкою. При використанні хмарного хостингу або віртуального сервера вартість обслуговування системи залишається відносно невисокою та залежить від кількості користувачів і навантаження на систему.

Впровадження розробленої платформи дозволяє автоматизувати процес створення та проведення тестування, скоротити час перевірки результатів, підвищити ефективність контролю знань та зменшити витрати на використання сторонніх сервісів. Це робить розробку програмного продукту економічно виправданою та перспективною для використання в освітніх закладах.

1.4 Очікувані результати

Результатом виконання кваліфікаційної роботи стане функціональна вебплатформа для створення та проведення тестування, яка забезпечуватиме можливість реєстрації користувачів, створення тестів, проходження тестувань та автоматичного оцінювання результатів. Платформа підтримуватиме різні ролі користувачів, роботу з питаннями різних типів, а також збереження результатів у базі даних.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		14

Очікується, що впровадження розробленої системи дозволить спростити процес контролю знань, підвищити ефективність організації тестування та забезпечити зручний доступ до навчальних матеріалів і статистики успішності користувачів.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
						15
Змн.	Арк.	№ докум.	Підпис	Дата		

2 Теоретична частина

2.1 Веборієнтовані системи тестування

У сучасних умовах розвитку інформаційних технологій веборієнтовні системи тестування набувають все більшого поширення в освітній сфері. Такі системи дозволяють автоматизувати процес контролю знань, спростити організацію перевірки навчальних досягнень та забезпечити швидкий доступ до результатів тестування через мережу Інтернет.

Веборієнтована система тестування являє собою програмний комплекс, який забезпечує створення, проведення та оцінювання тестових завдань за допомогою веббраузера. Основною перевагою такого підходу є відсутність необхідності встановлення спеціального програмного забезпечення на комп'ютери користувачів. Для роботи із системою достатньо мати доступ до мережі Інтернет та сучасний веббраузер.

Однією з головних переваг онлайн-тестування є можливість автоматичного оцінювання результатів. Це дозволяє значно скоротити час перевірки відповідей та підвищити об'єктивність оцінювання. Крім того, результати тестування можуть зберігатися в базі даних для подальшого аналізу успішності користувачів.

Сучасні системи тестування також забезпечують підтримку різних типів запитань, включаючи вибір однієї правильної відповіді, вибір декількох правильних відповідей та відкриті запитання. Додатково можуть використовуватися графічні матеріали, що дозволяє розширити можливості представлення навчальної інформації та зробити тестування більш наочним.

2.2 Клієнтські вебтехнології та бібліотека React

Клієнтські вебтехнології відіграють важливу роль у створенні сучасних вебзастосунків, оскільки саме вони відповідають за відображення інтерфейсу користувача та взаємодію з ним. До основних клієнтських технологій належать HTML, CSS та JavaScript. HTML використовується для формування структури вебсторінки, CSS відповідає за її зовнішній вигляд, а JavaScript забезпечує динамічну поведінку елементів інтерфейсу.

У сучасній веброзробці часто використовуються односторінкові застосунки, або SPA. Принцип роботи SPA полягає в тому, що вебсторінка не перезавантажується повністю під час переходу між розділами системи. Замість цього змінюється лише необхідна частина інтерфейсу, а дані завантажуються з сервера за допомогою HTTP-запитів. Такий підхід забезпечує швидшу роботу застосунку та більш зручну взаємодію користувача із системою.

Для реалізації клієнтської частини розробленої платформи було використано бібліотеку React. Вона дозволяє створювати інтерфейс за допомогою окремих компонентів, кожен із яких відповідає за певну частину сторінки або функціональності. Наприклад, у системі тестування окремими компонентами можуть бути навігаційна панель, сторінка входу, список тестів, форма створення тесту, сторінка проходження тестування та панель адміністратора.

Компонентний підхід спрощує розробку та підтримку програмного коду, оскільки окремі частини інтерфейсу можна змінювати незалежно одна від одної. Це особливо важливо для вебплатформи тестування, де система складається з різних сторінок і функціональних блоків для адміністратора, викладача та студента.

React також зручно поєднується з REST API, що дозволяє клієнтській частині обмінюватися даними із сервером. У межах даного проєкту клієнтська

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		17

частина надсилає запити до серверної частини авторизації користувачів, отримання тестів, створення питань, збереження відповідей та отримання результатів тестування.

2.3 Серверна розробка з використанням Spring Boot

Серверна частина вебзастосунку відповідає за обробку запитів користувачів, виконання бізнес-логіки, взаємодію з базою даних та передачу необхідних даних клієнтській частині. У розробленій платформі серверна частина є основою всієї системи, оскільки саме вона забезпечує роботу з користувачами, тестами, питаннями, відповідями та результатами проходження тестування.

Для реалізації серверної частини було використано фреймворк Spring Boot мовою програмування Java. Spring Boot дозволяє швидко створювати вебзастосунки та REST API, які використовуються для обміну даними між клієнтською та серверною частинами. REST API дає змогу клієнтській частині надсилати запити на сервер для отримання списку тестів, створення нових тестових завдань, проходження тестування, збереження відповідей і перегляд результатів.

Розроблена система побудована за принципом клієнт-серверної архітектури. Клієнтська частина відповідає за відображення інтерфейсу користувача та надсилання запитів до сервера. Серверна частина обробляє ці запити, виконує необхідні операції та повертає відповідь у зручному для клієнта форматі. Такий підхід дозволяє розділити логіку відображення і логіку обробки даних, що спрощує підтримку та розвиток програмного продукту.

Однією з переваг Spring Boot є зручна організація структури проєкту. У застосунку можна виділити окремі рівні: контролери, сервіси, репозиторії та моделі даних. Контролери приймають HTTP-запити, сервіси містять основну бізнес-логіку, репозиторії відповідають за взаємодію з базою даних, а моделі

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		18

описують основні сутності системи, такі як користувач, тест, питання, відповідь і результат.

Spring Boot також добре інтегрується з реляційними базами даних, зокрема з MySQL. Завдяки цьому система може зберігати інформацію про користувачів, створені тести, питання, варіанти відповідей, результати тестування та інші дані. Крім того, фреймворк дозволяє реалізувати механізми авторизації та обмеження доступу до кремих функцій залежно від ролі користувача.

2.4 Система керування базами даних MySQL

База даних є важливою частиною будь-якої інформаційної системи, оскільки вона забезпечує зберігання, обробку та доступ до даних. У вебплатформі для створення та проведення тестування база даних використовується для збереження інформації про користувачів, тести, питання, варіанти відповідей, результати проходження тестів та інші дані, необхідні для роботи системи.

Для реалізації зберігання даних у проєкті використовується система керування базами даних MySQL. Вона належить до реляційних баз даних, у яких інформація зберігається у вигляді таблиць. Кожна таблиця містить записи певного типу, наприклад користувачів, тести або результати. Такий підхід дозволяє впорядковано зберігати дані та швидко отримувати потрібну інформацію за допомогою запитів.

Реляційна модель даних добре підходить для розробленої платформи, оскільки між основними об'єктами системи існують логічні зв'язки. Наприклад, один викладач може створювати декілька тестів, кожен тест може містити багато питань, а кожне питання може мати кілька варіантів відповідей. Також результати тестування пов'язані зі студентом і конкретним тестом.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		19

Використання зв'язків між таблицями дозволяє уникнути дублювання інформації та забезпечити цілісність даних.

У межах проєкту база даних використовується для підтримки основної функціональності системи. Під час реєстрації та авторизації користувачів зберігаються їхні облікові записи та ролі. Під час створення тестів у базу даних записується інформація про назву тесту, предмет, автора, тип доступу та код доступу для приватних тестів. Окремо зберігаються питання, відповіді та результати проходження тестування.

MySQL також забезпечує надійність роботи системи завдяки підтримці первинних і зовнішніх ключів. Первинні ключі використовуються для унікальної ідентифікації записів у таблицях, а зовнішні ключі дозволяють встановлювати зв'язки між різними таблицями. Це особливо важливо для системи тестування, де потрібно правильно пов'язувати тести з викладачами, питання з тестами, відповіді з питаннями, а результати зі студентами.

2.5 Автентифікація та авторизація користувачів

Автентифікація та авторизація є важливими складовими безпеки вебзастосунків. Автентифікація використовується для перевірки особи користувача під час входу в систему, а авторизація визначає, які дії користувач має право виконувати після успішного входу. У вебплатформі для створення та проведення тестувань ці механізми необхідні для захисту даних і правильного розподілу доступу між різними категоріями користувачів.

У розробленій системі передбачено три основні ролі користувачів: адміністратор, викладач і студент. Адміністратор має доступ до керування користувачами та тестами. Викладач може створювати тести, додавати питання, переглядати результати студентів і керувати власними тестовими завданнями. Студент має можливість переглядати доступні тести, проходити тестування та переглядати власні результати.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
						20
Змн.	Арк.	№ докум.	Підпис	Дата		

Для реалізації автентифікації в системі використовується механізм JWT-токенів. Після успішного входу користувача сервер створює токен, у якому міститься інформація про користувача, зокрема його ідентифікатор та роль. Надалі цей токен використовується клієнтською частиною для надсилання захищених запитів до серверної частини. Сервер перевіряє токен і визначає, чи має користувач право виконувати відповідну дію.

Безпека вебзастосунку також залежить від правильного обмеження доступу до окремих маршрутів і функцій. Наприклад, студент не повинен мати доступу до створення тестів або керування користувачами, а викладач не повинен виконувати адміністративні дії. Для цього серверна частина перевіряє роль користувача перед виконанням запитів, а клієнтська частина відображає лише ті елементи інтерфейсу, які відповідають правам користувача.

2.6 Висновки до теоретичної частини

Обрані технології – React, Spring Boot, MySQL та JWT – забезпечують надійну основу для створення вебплатформи тестування. React дозволяє реалізувати зручний та інтерактивний користувацький інтерфейс, що забезпечує комфортну роботу студентів і викладачів із тестами, питаннями та результатами. Spring Boot забезпечує розробку серверної частини, обробку запитів, реалізацію бізнес-логіки та взаємодію з базою даних. MySQL використовується для структурованого зберігання інформації про користувачів, тести, питання, відповіді та результати проходження тестування. Використання JWT-токенів дозволяє організувати автентифікацію, авторизацію та розподіл доступу між ролями користувачів. Сукупність цих технологій забезпечує стабільну роботу платформи, безпеку даних, зручність використання та можливість подальшого розширення функціональних можливостей системи.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		21

3 Проектна частина

3.1 Проектування архітектури вебплатформи

Під час розробки вебплатформи для створення та проведення тестувань було обрано клієнт-серверну архітектуру. Такий підхід відповідає загальним принципам проектування програмного забезпечення, оскільки дозволяє розділити систему на окремі логічні частини та спростити її подальшу підтримку [13]. Клієнтська частина відповідає за відображення інтерфейсу користувача та взаємодію з ним, серверна частина виконує обробку запитів і бізнес-логіку, а база даних забезпечує зберігання всієї необхідної інформації.

Загальна схема роботи системи полягає в тому, що користувач виконує дію в інтерфейсі, наприклад входить у систему, створює тест або проходить тестування. Після цього клієнтська частина надсилає HTTP-запит до серверної частини. Сервер обробляє запит (лістинг 3.1), за потреби звертається до бази даних, після чого повертає відповідь клієнтській частині. Отримані дані відображаються користувачу у вебінтерфейсі.

На рисунку 3.1 наведено схему взаємодії клієнтської та серверної частини вебплатформи.

Схема взаємодії клієнтської та серверної частини вебплатформи

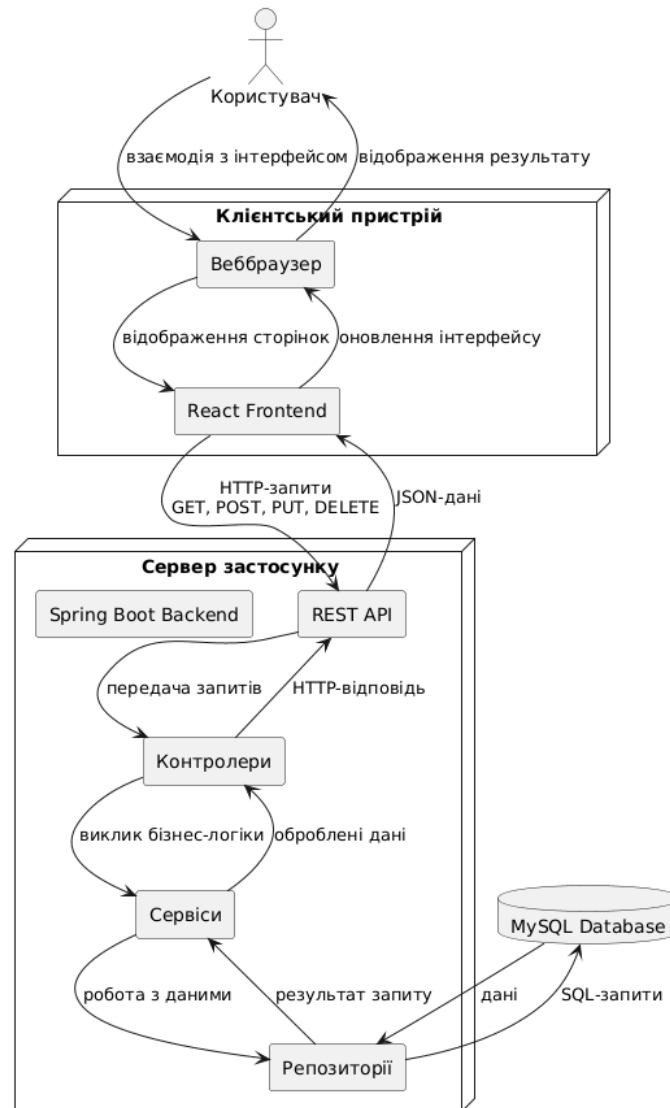


Рисунок 3.1 – Схема взаємодії клієнтської та серверної частини

Для прикладу взаємодії клієнтської частини із сервером можна навести фрагмент коду, у якому React-застосунок надсилає запит до серверної частини для отримання списку тестів.

Лістинг 3.1 – Надсилення запиту до серверної частини

```

const loadTests = async () => {
  try {

```

Продовження лістингу 3.1

```
const res = await API.get('/tests');
setTests(res.data);
} catch (err) {
console.log(err);
alert('Помилка при завантаженні тестів');
}
};
```

У цьому прикладі клієнтська частина звертається до REST API серверної частини, отримує список доступних тестів і зберігає його у стані компонента для подальшого відображення на сторінці.

Обрана архітектура дозволяє зробити систему гнучкою та зручною для подальшого розвитку. За потреби можна окремо змінювати клієнтську частину, серверну логіку або структуру бази даних, не порушуючи роботу всієї платформи.

3.2 Проєктування бази даних

Для зберігання даних вебплатформи було використано реляційну базу даних MySQL. Під час проєктування структури бази даних було визначено основні сутності системи: користувачі, тести, питання, відповіді та результати проходження тестування.

Основною таблицею системи є таблиця користувачів, у якій зберігається інформація про зареєстрованих користувачів платформи. Для кожного користувача зберігається його ідентифікатор, електронна пошта, пароль та роль у системі.

Таблиця тестів використовується для зберігання інформації про створенні тести. Кожен тест має назву, предмет, тип доступу та посилання на викладача, який його створив. Для приватних тестів додатково зберігається код доступу та час завершення його дії. Це дозволяє реалізувати можливість проходження окремих тестів лише за спеціальним кодом.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
						24
Змн.	Арк.	№ докум.	Підпис	Дата		

Питання тесту зберігаються в окремій таблиці. Кожне питання належить до певного тесту та має власний тип: вибір однієї правильної відповіді, вибір декількох правильних відповідей або відкрита відповідь. Також для питання може зберігатися посилання на зображення, якщо викладач додає графічний матеріал до завдання.

Таблиця відповідей містить варіанти відповідей для тестових питань. Кожна відповідь пов'язана з конкретним питанням і має ознаку правильності. Завдяки цьому система може автоматично перевіряти відповіді студента після завершення тестування.

Окремо використовується таблиця результатів, у якій зберігаються підсумки проходження тестів студентами. Результат пов'язаний із конкретним студентом і тестом, а також містить отриману кількість балів. Це дозволяє студенту переглядати власну успішність, а викладачу – контролювати результати проходження його тестів.

Зв'язки між таблицями (рисунок 3.2) побудовані таким чином, щоб забезпечити цілісність даних. Один викладач може створити багато тестів, один тест може містити багато питань, кожне питання може мати декілька відповідей, а один студент може мати багато результатів проходження тестів. Така структура відповідає логіці роботи платформи та забезпечує зручне зберігання й обробку інформації.

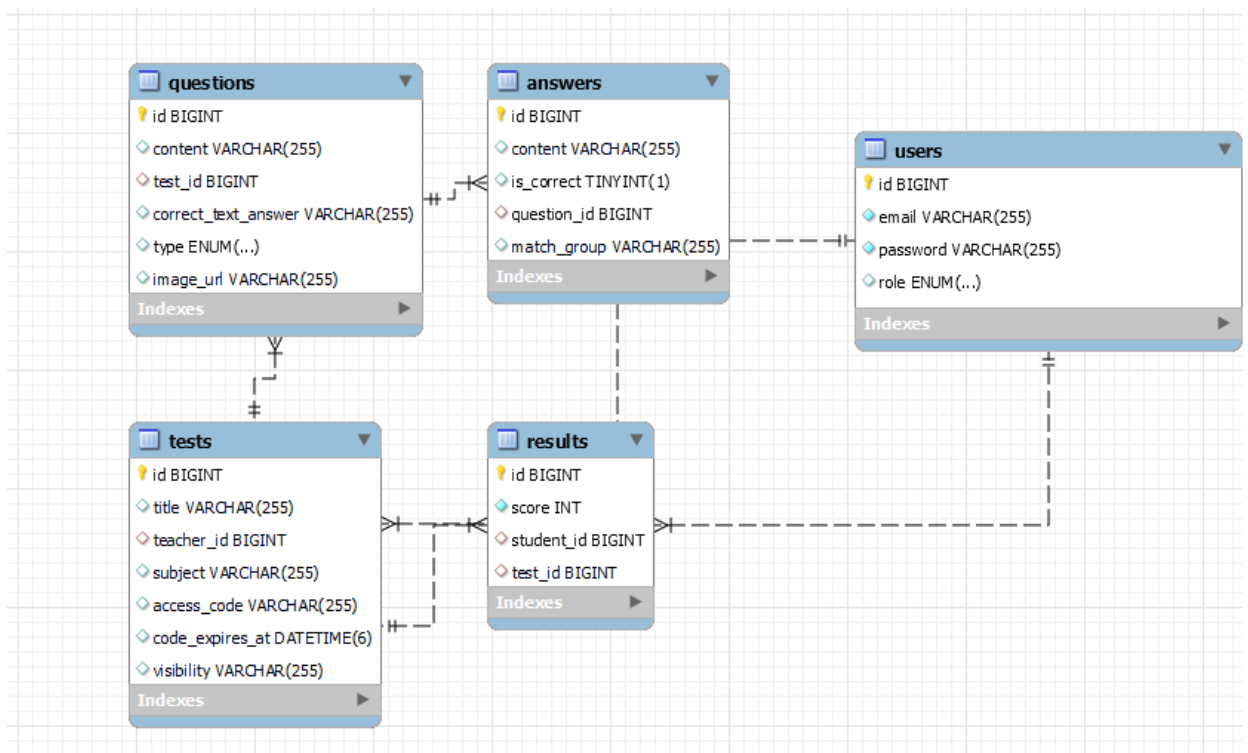


Рисунок 3.2 – ER-діаграма бази даних вебплатформи

3.3 Реалізація серверної частини

Серверна частина вебплатформи реалізована за допомогою фреймворку Spring Boot. Вона відповідає за обробку запитів від клієнтської частини, виконає бізнес-логіку, роботу з базою даних, авторизацію користувачів, створення тестів, додавання питань, збереження відповідей та обробку результатів проходження тестування.

Структура серверної частини побудована за багаторівневим принципом. Основними елементами є моделі, репозиторії, сервіси та контролери. Моделі описують основні сутності системи. Репозиторії використовуються для взаємодії з базою даних. Сервіси містять основну бізнес-логіку, а контролери приймають HTTP-запити від клієнтської частини та повертають відповідні результати.

Для реалізації взаємодії між клієнтською та серверною частинами використовується REST API. Клієнтська частина надсилає запити до відповідних маршрутів серверної частини. Сервер обробляє ці запити, звертається до сервісів і репозиторіїв, після чого повертає відповідь у форматі JSON.

У серверній частині реалізовано кілька основних контролерів. Контролер користувачів відповідає за роботу з обліковими записами, контролер тестів – за створення, отримання та видалення тестів, контролер питань – за додавання питань до тесту, контролер відповідей – за збереження варіантів відповідей, а контролер результатів – за обробку та збереження результатів проходження тестування.

Окрему роль у системі відіграє механізм авторизації. Для входу користувача в систему використовується перевірка облікових даних, після чого створюється JWT-токен. Цей токен використовується під час подальших запитів до захищених ресурсів системи. Завдяки цьому реалізовано розподіл доступу між ролями.

Також у серверній частині реалізовано можливість створення приватних тестів. Якщо викладач обирає приватний тип доступу, система генерує спеціальний код доступу та встановлює час його дії. Студент може перейти до проходження такого тесту лише після введення правильного та дійсного коду.

Для питань тесту передбачено можливість завантаження зображень. Під час створення питання сервер приймає файл зображення, зберігає його у відповідній папці на сервері та записує шлях до файлу в базу даних. Після цього клієнтська частина може отримати це посилання та відобразити зображення під час проходження тесту.

На рисунку 3.3 показано зв'язки основних сутностей вебплатформи:

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		27

Діаграма класів основних сутностей вебплатформи

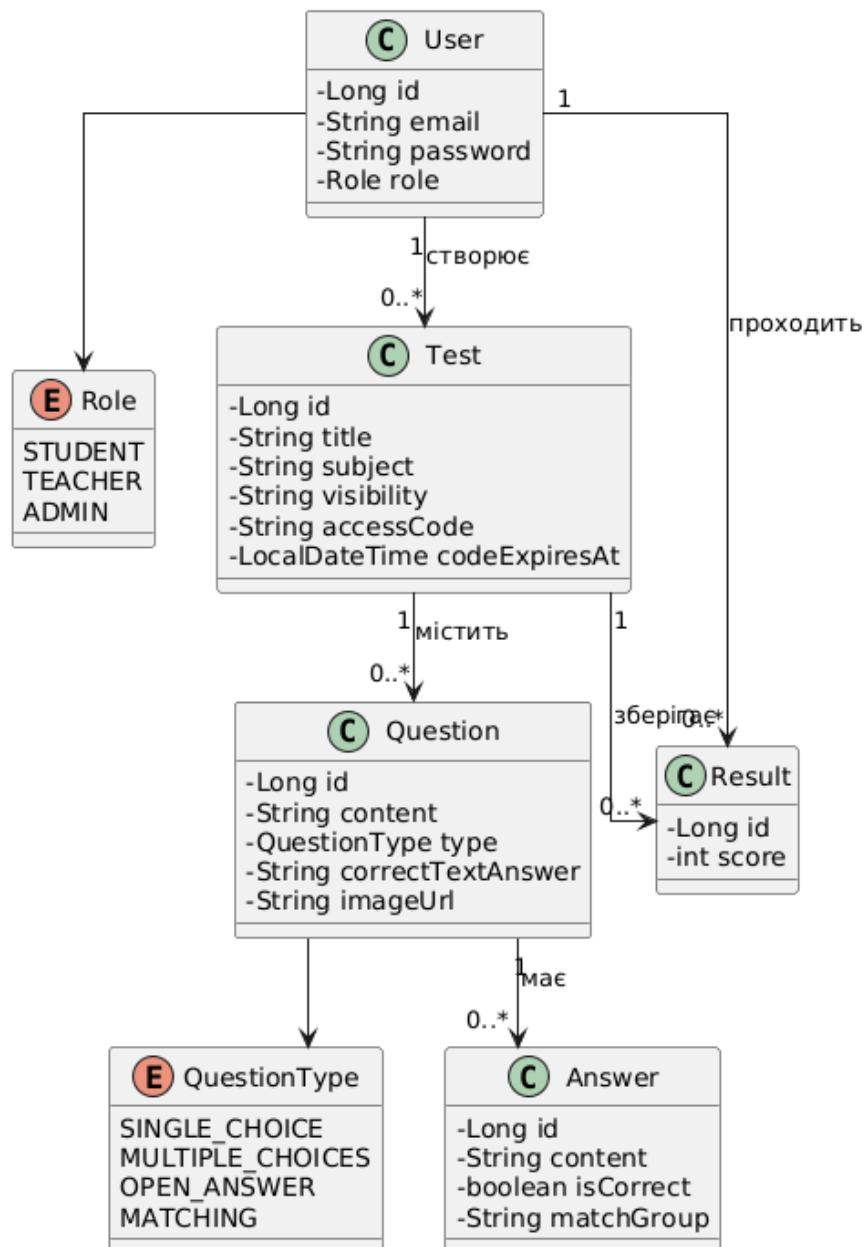


Рисунок 3.3 – Діаграма класів основних сутностей вебплатформи

Для прикладу реалізації серверної логіки можна навести фрагмент методу створення тесту (лістинг 3.2). У ньому відбувається пошук викладача, створення нового тесту та збереження його в базі даних.

Лістинг 3.2 – Фрагмент створення тесту на серверній частині

```
public TestResponse createTest(CreateTestRequest request){  
  
    User      teacher      =      userRepository.findById  
(request.getTeacherId()).orElseThrow(()      ->      new  
RuntimeException('Teacher not found'));  
    Test.TestBuilder builder = Test.builder()  
        .title(request.getTitle())  
        .subject(request.getSubject())  
        .visibility(request.getVisibility())  
        .createdBy(teacher);  
    if('PRIVATE'.equals(request.getVisibility())){  
        builder.accessCode(generateCode());  
        builder.codeExpiresAt(LocalDateDateTime.now()  
            .plusMinutes(request.getCodeLifetimeMinutes()));  
    }  
    Test saved = testRepository.save(builder.build());  
    return TestResponse.builder()  
        .id(saved.getId())  
        .title(saved.getTitle())  
        .subject(saved.getSubject())  
        .teacherId(saved.getCreatedBy().getId())  
        .build()  
    }  
}
```

У наведеному фрагменті показано, як серверна частина отримує дані з клієнтського запиту, створює об'єкт тесту, пов'язує його з викладачем та зберігає в базі даних. Після збереження сервер повертає клієнтській частині відповідь із даними створеного тесту.

3.4 Реалізація клієнтської частини

Клієнтська частина вебплатформи реалізована за допомогою бібліотеки React. Вона відповідає за відображення інтерфейсу користувача, взаємодію з формами, перехід між сторінками, надсилання запитів до серверної частини та відображення отриманих даних. Для організації навігації між сторінками використовується маршрутизація, що дозволяє користувачам переходити між головною сторінкою, сторінкою входу, реєстрації, панелями користувачів і сторінками проходження тестування.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		29

Для переходу між сторінками використовується маршрутизація React Router. Вона дозволяє організувати окремі маршрути для різних сторінок. Завдяки цьому користувач може переміщуватися між розділами платформи без повного перезавантаження сторінки.

Для обміну даними між клієнтською та серверною частинами використовується бібліотека Axios. У проєкті створено окремий файл налаштування API (лістинг 3.3), у якому вказано базову адресу серверної частини та реалізовано автоматичне додавання JWT-токена до заголовків запитів. Це дозволяє надсилати захищені запити до серверної частини після авторизації користувача.

Лістинг 3.3 – Налаштування Axios для взаємодії з серверною частиною

```
import axios from 'axios';
const API = axios.create({
  baseURL: 'http://localhost:8080',
});
API.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if(token){
    config.headers.Authorization = 'Bearer ${token}';
  }
  return config;
});
export function parseJwt(token){
  if(!token) return null;
  return JSON.parse(atob(token.split('.')[1]));
}
export default API;
```

У наведеному лістингу 3.3 створюється об'єкт API, який використовується для надсилання запитів до серверної частини. За допомогою перехорлювача запитів до кожного захищеного запиту автоматично додається JWT-токен, який зберігається в локальному сховищі браузера після входу користувача в систему. Функція parseJwt використовується для отримання інформації з токена, зокрема ролі та ідентифікатор користувача.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
						30
Змн.	Арк.	№ докум.	Підпис	Дата		

Клієнтська частина складається з окремих сторінок та компонентів. Головна сторінка (рисунки 3.4) призначена для переходу до входу або реєстрації. Сторінки входу (рисунки 3.5) та реєстрації забезпечують авторизацію користувачів у системі. Після успішного входу користувач переходить до відповідного розділу залежно від своєї ролі.

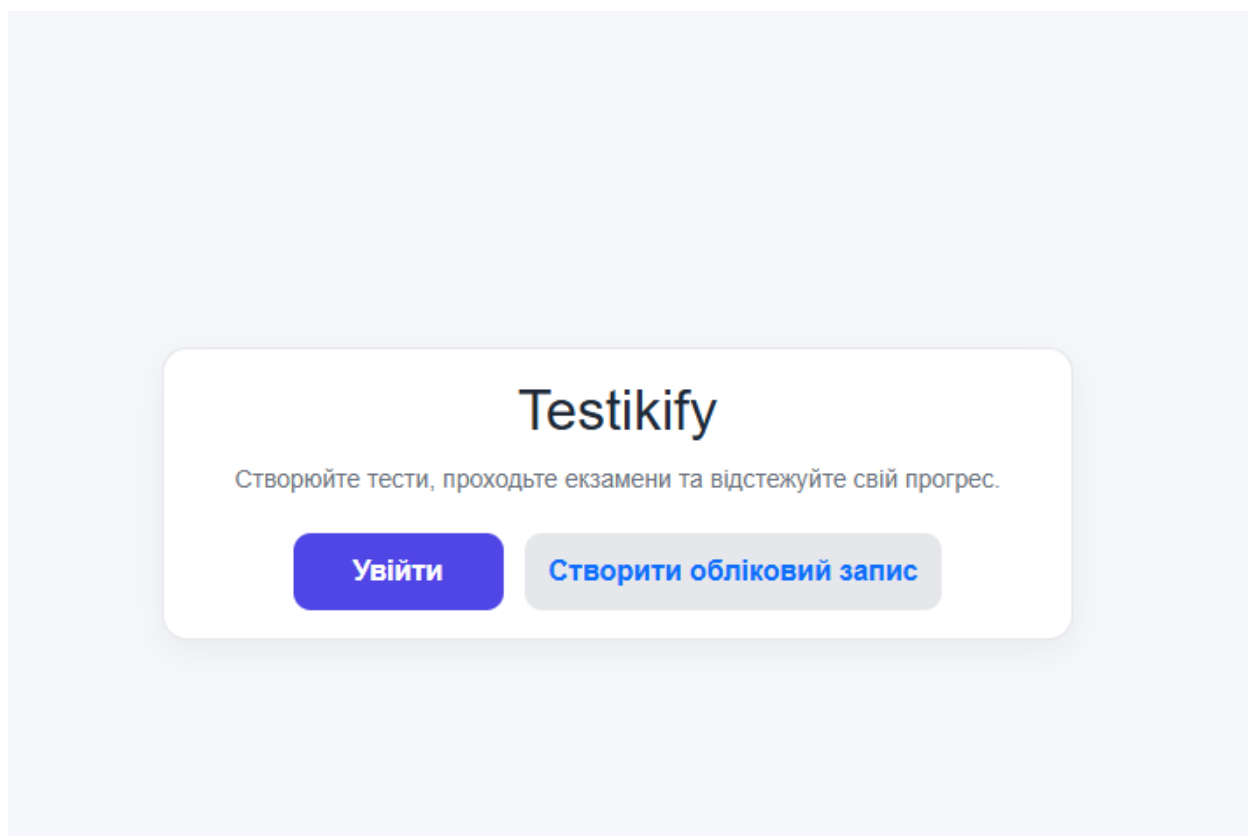


Рисунок 3.4 – Головна сторінка вебплатформи

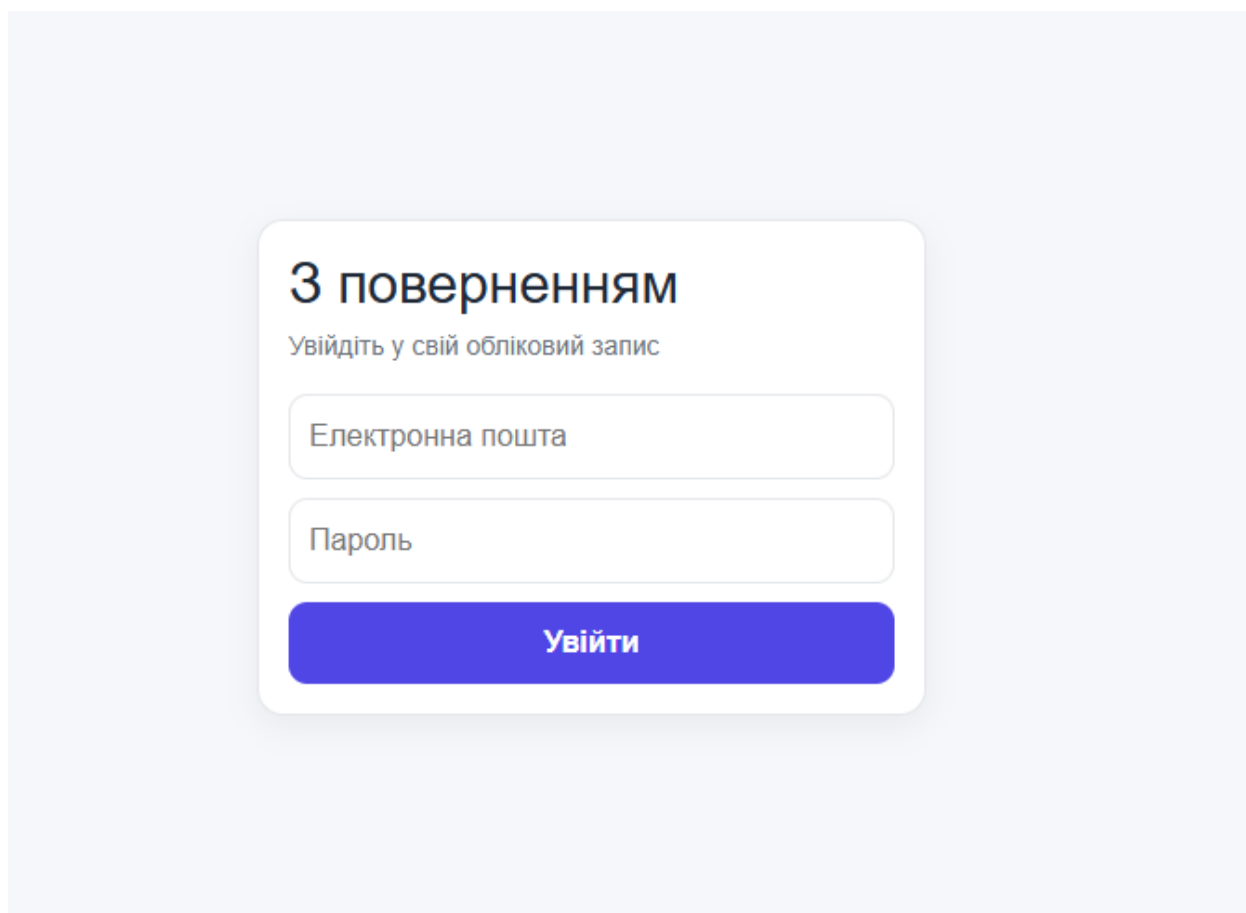


Рисунок 3.5 – Сторінка входу користувача

Для адміністратора реалізовано окрему панель керування (рисунок 3.6). У ній адміністратор може переглядати список користувачів, створювати нові облікові записи, редагувати ролі користувачів та видаляти користувачів або тести. Такий інтерфейс дозволяє здійснювати базове адміністрування платформи без прямого доступу до бази даних.

Панель викладача (рисунок 3.7) призначена для створення та керування тестами. Викладач може ввести назву тесту, обрати предмет, визначити тип доступу до тесту та створити приватний тест із кодом доступу. Також у панелі викладача реалізовано додавання питань, варіантів відповідей, позначення правильної відповіді, завантаження зображень до питань, перегляд уже створених питань і результатів студентів.

Панель адміна

Створити обліковий запис

Електронна пошта

Пароль

Студент

Створити

Користувачі

admin@gmail.com
ADMIN

Редагувати

Видалити

teacher2@gmail.com
TEACHER

Редагувати

Видалити

teacher3@example.com
TEACHER

Редагувати

Видалити

student
STUDENT

Редагувати

Видалити

student2
ADMIN

Редагувати

Видалити

Тести

First test
Автор: admin@gmail.com

Видалити тест

myFirstTest
Автор: admin@gmail.com

Видалити тест

myFirstTest
Автор: admin@gmail.com

Видалити тест

teacher2Test
Автор: teacher2@gmail.com

Видалити тест

testMath
Автор: teacher2@gmail.com

Видалити тест

testHistory
Автор: teacher2@gmail.com

Видалити тест

testTest
Автор: teacher2@gmail.com

Видалити тест

testMath2
Автор: teacher2@gmail.com

Видалити тест

Physics
Автор: teacher3@example.com

Видалити тест

Img testing 2
Автор: teacher3@example.com

Видалити тест

Рисунок 3.6 – Панель адміністратора

Головна
Панель вчителя
Тести
Приєднатися до тесту за кодом

Пошук тестів...

Пошук

Вийти

Панель вчителя

Створити тест

Назва тесту

Інше

Відкритий для всіх

Створити тест

Physics

Physics

Тип: PUBLIC

Видалити тест

PrivateTest

Other

Тип: PRIVATE

Код доступу: ABFDA7C7

Скопіювати код

Видалити тест

Img testing 2

Other

Тип: PUBLIC

Видалити тест

Code testing

Other

Тип: PRIVATE

Код доступу: 2A673649

Скопіювати код

Видалити тест

Рисунок 3.7 – Панель викладача

Для користувачів реалізовано сторінку зі списком доступних тестів (рисунок 3.8). На цій сторінці студент може переглядати відкриті тести, фільтрувати їх за предметом та переходити до проходження вибраного тесту. Окремо створено сторінку введення коду доступу (рисунок 3.9), яка використовується для проходження приватних тестів. Якщо код правильний і термін його дії не завершився, студент отримує доступ до відповідного тесту.

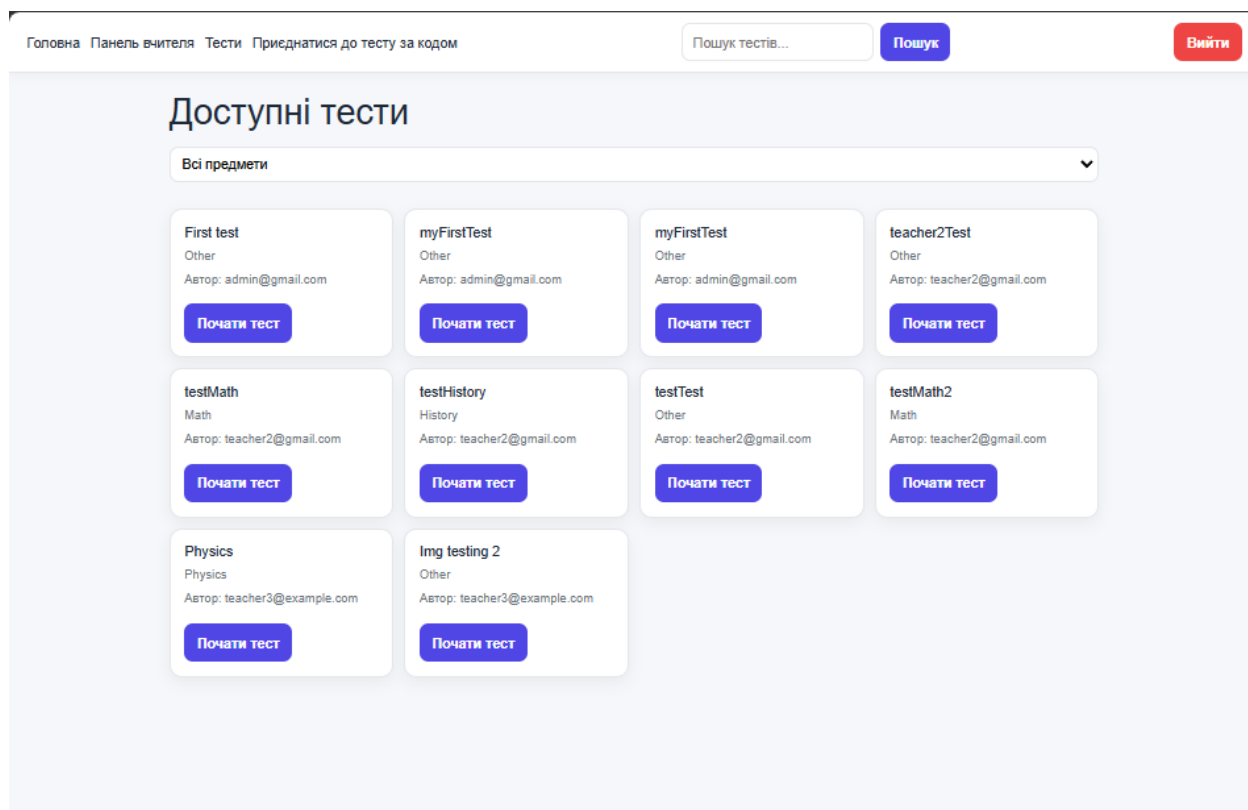


Рисунок 3.8 – Сторінка списку доступних тестів

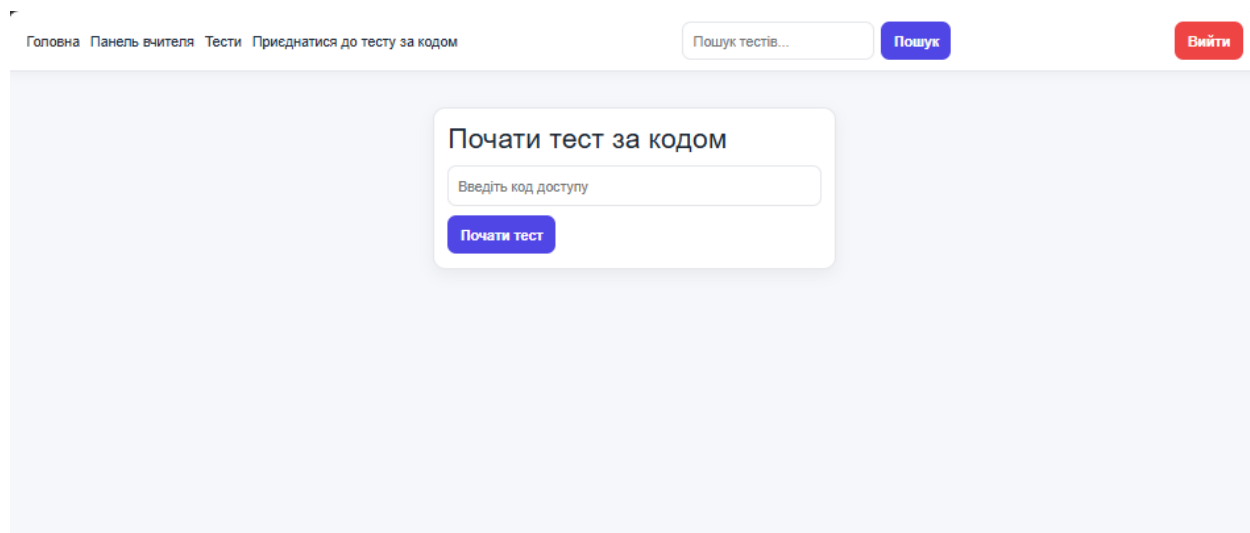


Рисунок 3.9 – Сторінка введення коду доступу

Сторінка проходження тесту (рисунок 3.10) відображає поточне питання, варіанти відповідей, прогрес проходження та кнопку переходу до

наступного питання або завершення тесту. Якщо питання містить графічний матеріал, зображення також відображається на сторінці. Після завершення тестування відповіді студента надсилаються на серверну частину, де виконується перевірка та збереження результату.

Рисунок 3.10 – Сторінка проходження тестування

Для прикладу реалізації клієнтської логіки можна навести фрагмент коду (лістинг 3.4), який відповідає за перехід студента до приватного тесту за кодом доступу.

Лістинг 3.4 – Перехід до приватного тесту за кодом доступу

```
const join = async () => {
  try {
    const res = await API.post('/tests/join?code=${code}');
    navigate('/test/${res.data.id}');
  } catch {
    alert('Неправильний код доступу або час проходження вже минув');
  }
};
```

У наведеному фрагменті клієнтська частина надсилає код доступу на сервер. Якщо код є правильним і термін його дії не завершився, користувач перенаправляється на сторінку проходження відповідного тесту. У разі помилки система повідомляє користувача про недійсний або прострочений код.

Під час оформлення інтерфейсу користувача використовувалися каскадні таблиці стилів SCSS. Стили застосовувалися для створення єдиного зовнішнього вигляду сторінок, оформлення карток, кнопок, форм, навігаційної панелі та адаптації елементів інтерфейсу до різних розмірів екрана. Для оптимізації частини стилів та покращення структури SCSS-коду також використовувалися можливості інструмента штучного інтелекту GPT-5.5, що дозволило швидше підібрати зручні варіанти оформлення та виправити проблеми з відступами й адаптивністю інтерфейсу.

3.5 Реалізація системи ролей та доступу

У вебплатформі для створення та проведення тестувань реалізовано систему ролей користувачів. Вона необхідна для того, щоб кожен користувач мав доступ лише до тих функцій, які відповідають його призначенню в системі.

Адміністратор має найширші права доступу. Викладач має доступ до функцій створення та керування тестами. Студент має обмежений доступ до платформи.

У серверній частині для забезпечення безпеки використовується Spring Security. Даний інструмент дозволяє налаштовувати доступ до окремих маршрутів, перевіряти авторизованих користувачів і захищати ресурси системи від несанкціонованого доступу. Завдяки цьому функції адміністратора, викладача та студента можуть бути розмежовані відповідно до їхніх ролей.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
						37
Змн.	Арк.	№ докум.	Підпис	Дата		

На клієнтській частині також реалізовано перевірку ролі користувача. Після авторизації роль зберігається в локальному сховищі браузера, а інтерфейс відображає лише ті елементи навігації, які відповідають правам користувача.

На рисунку 3.11 показано функції які можуть використовувати різні особи, з різними ролями.

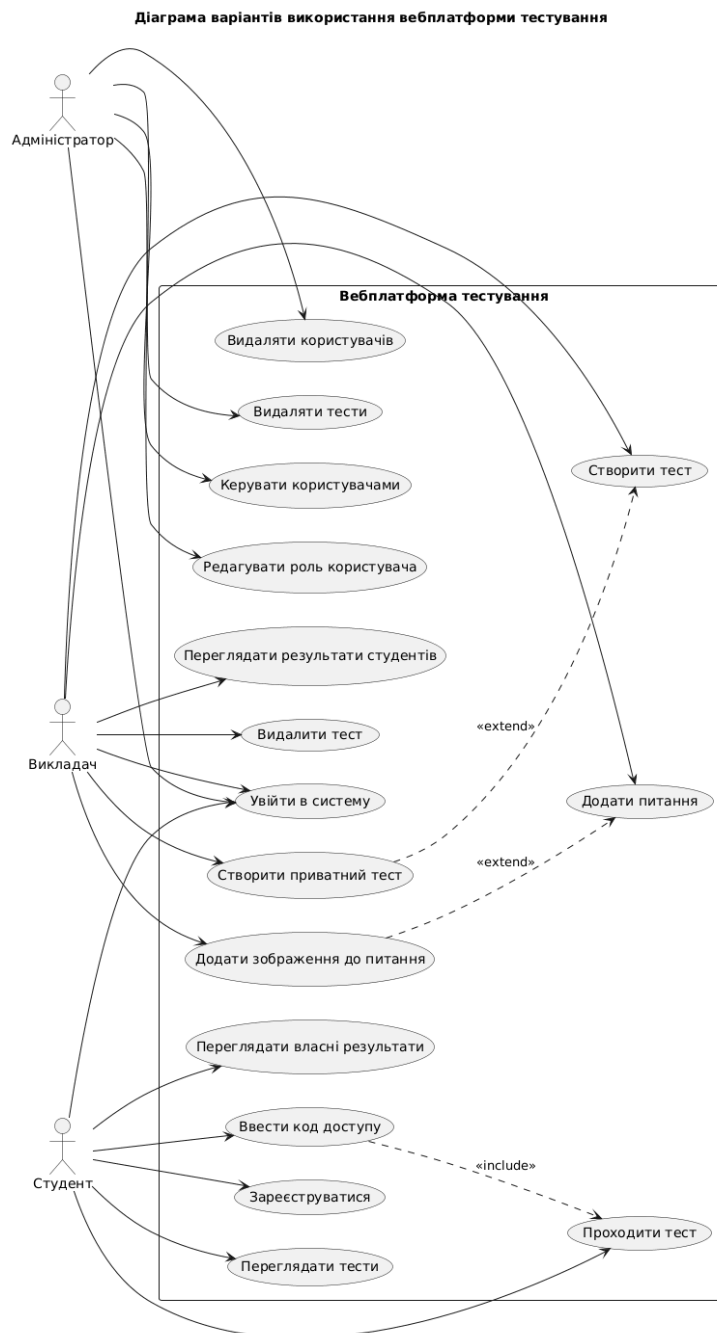


Рисунок 3.11 – Діаграма варіантів використання вебплатформи

Для прикладу реалізації ролей у системі можна навести фрагмент переліку ролей користувачів (лістинг 3.5).

Лістинг 3.5 – Перелік ролей користувачів

```
public enum Role{  
    STUDENT,  
    TEACHER,  
    ADMIN  
}
```

У наведеному фрагменті показано три ролі, які використовуються в системі для розмежування доступу. Кожна роль визначає набір функцій, доступних користувачу після входу в систему.

3.6 Реалізація тестування та обробки результатів

Однією з основних функцій вебплатформи є проходження тестування студентами (рисунок 3.12) та автоматична обробка результатів. Даний процес охоплює створення тесту викладачем, додавання питань і варіантів відповідей, вибір відповідей студентом, надсилання результатів на серверну частину та збереження отриманої оцінки в базі даних.

Створення тесту виконується у панелі викладача. Викладач вводить назву тесту, обирає предмет і визначає тип доступу до тесту. Після створення тесту до нього можна додавати питання. Для кожного питання вказується текст завдання, тип питання та, за потреби, графічний матеріал. Після створення питання викладач додає варіанти відповідей і позначає правильні варіанти.

Під час проходження тесту студент послідовно переглядає питання та обирає відповіді. Клієнтська частина зберігає вибрані студентом варіанти у стані компонента. Для питань з однією правильною відповіддю зберігається

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		39

один ідентифікатор відповіді, а для питань з декількома правильними відповідями – список вибраних ідентифікаторів. Якщо питання має відкриту відповідь зберігається текст, введений студентом.

Після завершення тестування клієнтська частина надсилає на сервер ідентифікатор тесту, ідентифікатор студента та список відповідей. Серверна частина отримує всі питання відповідного тесту з бази даних і порівнює їх із відповідями студента. За кожну правильну відповідь студент отримує один бал. Після завершення перевірки формується результат, який зберігається в таблиці результатів.

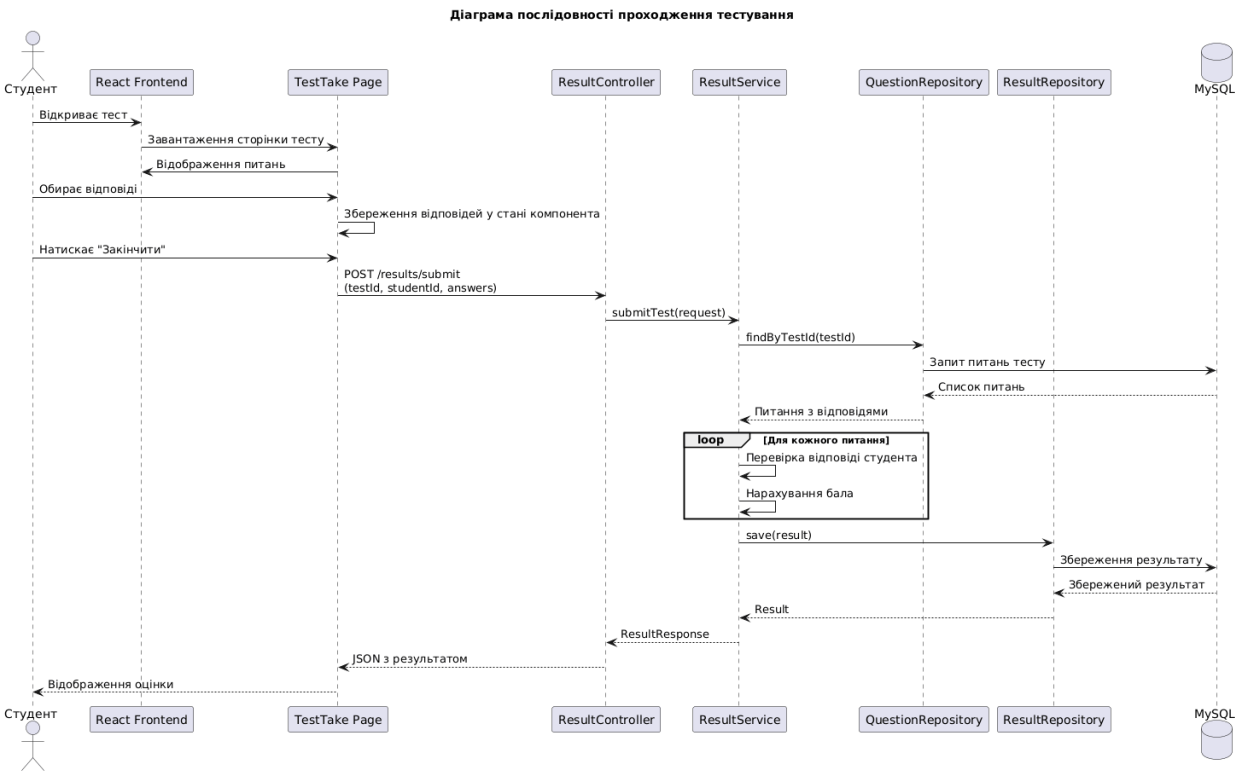


Рисунок 3.12 – Діаграма послідовності проходження тестування

Для прикладу реалізації автоматичного оцінювання можна навести фрагмент методу submitTest (лістинг 3.6), який відповідає за перевірку відповідей студента.

Лістинг 3.6 – Фрагмент автоматичного оцінювання результатів тестування

```

public ResultResponse submitTest (SubmitTestRequest
request) {
    List<Question> questions =
questionRepository.findByTestId(request.getTestId());
    int score = 0;
    for (Question q : questions) {
        AnswerSubmission sub =
request.getAnswers().stream().filter(a ->
a.getQuestionId().equals(q.getId())).findFirst().orElse(null);
        if (sub == null) {
            continue;
        }
        switch (q.getType()) {
            case SINGLE_CHOICE:
                Answer correctAnswer =
q.getAnswers().stream().filter(Answer::isCorrect).findFirst().or
ElseThrow(() -> new RuntimeException('No correct answer for
question ' + q.getId()));
                Long correctId = correctAnswer.getId();
                if (sub.getSelectedAnswerIds().contains(correctId)) {
                    score++;
                }
                break;
            case MULTIPLE_CHOICES:
                List<Long> correct =
q.getAnswers().stream().filter(Answer::isCorrect).map(Answer::ge
tId).toList();
                if (correct.equals(sub.getSelectedAnswerIds())) {
                    score++;
                }
                break;
            case OPEN_ANSWER:
                if (q.getCorrectTextAnswer().equalsIgnoreCase(sub.getTextAns
wer())) {
                    score++;
                }
                break;
        }
    }
    return saveResult(request.getStudentId(),
request.getTestId(), score);
}

```

Реалізований механізм автоматичного оцінювання дозволяє скоротити час перевірки робіт, зменшити вплив людського фактора та забезпечити швидку отримання результатів після завершення тестування.

3.7 Реалізація приватних тестів і завантаження зображень

У вебплатформі реалізовано можливість створення приватних тестів. Під час створення тесту викладач може обрати тип доступу. Якщо вибрано приватний тест, серверна частина автоматично генерує унікальний код і встановлює час завершення його дії. Це дозволяє викладачу надати доступ до тесту лише певним студентам і обмежити час його проходження.

Для генерації коду доступу використовується випадковий ідентифікатор, який скорочується до зручного для введення формату. Під час спроби входу за кодом сервер перевіряє, чи існує тест із таким кодом і чи не завершився термін його дії. Якщо код правильний, студент переходить до проходження тесту. Якщо код неправильний або прострочений, система виводить повідомлення про помилку. Для прикладу реалізації приватного тесту можна навести фрагмент методу генерації коду доступу (лістинг 3.7).

Лістинг 3.7 – Генерація коду доступу до приватного тесту

```
private String generateCode() {  
    return          UUID.randomUUID().toString().substring(0,  
8).toUpperCase();  
}
```

У наведеному фрагменті використовується клас UUID, який створює випадковий унікальний ідентифікатор. Для зручності користувача з нього беруться перші вісім символів, після чого вони переводяться у верхній регістр. Отриманий код зберігається в базі даних і використовується студентом для доступу до приватного тесту.

Також у платформі реалізовано додавання зображень до питань. Під час створення питання викладач може завантажити графічний файл із комп'ютера. Серверна частина зберігає файл у спеціальній папці, формує шлях до нього та записує цей шлях у базу даних разом із питанням. Під час проходження тесту клієнтська частина отримує посилання на зображення його разом із текстом питання.

3.8 Висновки до проєктної частини

У проєктній частині було описано процес розробки вебплатформи для створення та проведення тестувань. Було спроектовано архітектуру системи, структуру бази даних, серверну та клієнтську частини застосунку. Також було розглянуто реалізацію системи ролей і доступу, механізм проходження тестування, автоматичне оцінювання результатів, створення приватних тестів за кодом доступу та можливість додавання зображень до питань. Реалізовані модулі забезпечують основну функціональність платформи та дозволяють використовувати її для організації контролю знань в освітньому процесі.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		43

4 Економічна частина

4.1 Характеристика роботи та її призначення

Програмний продукт призначений для використання в освітніх закладах з метою автоматизації процесу контролю знань студентів, спрощення роботи викладачів та забезпечення зручного доступу до тестових матеріалів через веббраузер.

Основним призначенням платформи є підвищення ефективності організації тестування в навчальному процесі. Використання такої системи дозволяє зменшити витрати часу на перевірку відповідей, забезпечити автоматичне оцінювання результатів, зберігати дані в базі даних та швидко отримувати інформацію про успішність студентів.

З економічної точки зору розробка платформи є доцільною, оскільки для її створення використовуються доступні програмні засоби та технології, зокрема React, Spring Boot, MySQL, Visual Studio Code та IntelliJ IDEA Community Edition. Це зменшує витрати на придбання ліцензійного програмного забезпечення та дозволяє створити функціональний програмний продукт із можливістю подальшого розвитку.

4.2 Розрахунок трудомісткості створення вебплатформи

Вартість створення вебплатформи включає витрати на обладнання, ліцензійне програмне забезпечення, оплату праці розробників, амортизацію обладнання, електроенергію та інші супровідні витрати. Розрахунок виконується для оцінки реальної вартості розробки програмного продукту.

Під час розрахунку враховано проєктування системи, розробку клієнтської частини, створення серверної логіки, проєктування бази даних, тестування та налагодження платформи. Загальні витрати визначено за формулами, наведеними в додатку Г.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		44

Для розробки використовуються безкоштовні засоби для розробки, отже витрати на ліцензійне програмне забезпечення мінімізовано.

Розрахунок трудомісткості створення вебплатформи виконано в Microsoft Excel. Результати наведено на рисунку 4.1.

SUM

✕

✓

fx

=(C2+4 * D2 + E2) / 6

Book1.xlsx

	A	B	C	D	E	F	G	H	I	J
1	№	Етап роботи	О, год	М, год	Р, год	Е, год	SD			
2	1	Аналіз вимог і постановка задачі	8	12	16	=(C2+4 * D2 + E2) / 6	1.33			
3	2	Проектування архітектури системи	10	14	20		1.67			
4	3	Проектування бази даних	8	12	18	12.33	1.67			
5	4	Розробка серверної частини	35	45	60	45.83	4.17			
6	5	Розробка клієнтської частини	35	50	70	50.83	5.83			
7	6	Реалізація авторизації та ролей	12	16	24	16.67	2			
8	7	Реалізація тестування і результатів	20	28	40	28.67	3.33			
9	8	Приватні тести та завантаження зображень	12	18	28	18.67	2.67			
10	9	Тестування та налагодження	20	30	45	30.83	4.17			
11	10	Підготовка документації	20	28	40	28.67	3.33			
12		Разом	180	253	361	258.8	30.17			
13										
14										

Рисунок 4.1 – Розрахунок трудомісткості створення вебплатформи

4.3 Розрахунок витрат на оплату праці розробника

Розрахунок витрат на оплату праці розробника виконано на основі трудомісткості створення вебплатформи. За результатами розрахунку трудомісткість становить 258,8 год.

Для виконання робіт передбачається участь одного розробника, який виконує проектування, розробку клієнтської та серверної частини, налаштування бази даних, тестування та налагодження системи. Годинну ставку розробника прийнято на рівні 200 грн/год.

Фактична заробітна плата розраховується як добуто годинної ставки на трудомісткість виконання робіт:

$$ЗП^к = 258,8 \cdot 200 = 51760 \text{ грн.}$$

Додаткова заробітна плата розраховується як добуто годинної ставки на трудомісткість виконання робіт:

$$ЗП_{\text{дод}} = 51760 \cdot 0,15 = 7764 \text{ грн.}$$

Нарахування на фонд оплати праці приймаються у розмірі 22% від суми фактичної та додаткової заробітної плати

$$Кзп = (51760 + 7764) \cdot 0,22 = 13095,28 \text{ грн.}$$

Загальні витрати на оплату праці розробника становлять:

$$З1 = 51760 + 7764 + 13095,28 = 72589,28 \text{ грн.}$$

Результати розрахунку витрат заробітної плати розробника наведено на рисунку 4.2

G20 $\times$ $\checkmark$ f_x $= (E20 + F20) * 22\%$									
Book1.xlsx * $\times$									
	A	B	C	D	E	F	G	H	I
16									
17									
18									
19	Посада	Кількість працівників	Трудомісткість, год	Оплата за годину, грн	Основна зарплата, грн	Додаткова зарплата 15%, грн	Кзп 22%, грн	Разом, грн	
20	Full-stack developer	1	258.8	200	51760	7764	13095.28	72619.28	
21	Всього		258.8		51760	7764	13095.28	72619.28	
22									
23									
24									
25									

Рисунок 4.2 – Розрахунок витрат заробітної плати розробника

4.4 Амортизаційні відрахування та витрати на електроенергію

Розрахунок собівартості комп'ютерного часу для створення вебплатформи виконується з урахуванням амортизаційних відрахувань, витрат на електроенергію та технічного обслуговування обладнання. Виходячи з трудомісткості роботи, тривалість використання комп'ютера становить 258,8 год.

Для розробки програмного продукту використовується персональний комп'ютер або ноутбук. Балансова вартість обладнання приймається 29000 грн, ліквідаційна вартість – 3000 грн, термін корисного використання – 36 місяців. Амортизаційні відрахування визначено за формулою з додатку Г:

$$A = (29000 - 3000) \div 36 = 722,22 \frac{\text{грн}}{\text{міс}}$$

Місячний фонд роботи обладнання приймається 170 год. Тоді амортизаційні відрахування за одну годину роботи становлять:

$$CA = 722,22 \div 170 = 4,25 \frac{\text{грн}}{\text{год}}$$

Витрати на електроенергію визначено за формулою додатку Г. Для розрахунку приймаємо потужність ноутбука 0,06 кВт, а вартість електроенергії – 4,32 грн/кВт · год (станом на 2026 рік):

$$CE = 0,06 \cdot 4,32 = 0,26 \frac{\text{грн}}{\text{год}}$$

Періодичність обслуговування визначено за формулою додатку Г:

$$\lambda = 1 \div 170 = 0,0059$$

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		47

Якщо годинна оплата працівника, що обслуговує обладнання, становить 200 грн/год, тоді витрати на технічне обслуговування становлять:

$$C_{TO} = 200 \cdot 0,0059 = 1,18 \frac{\text{грн}}{\text{год}}.$$

Собівартість однієї години роботи комп'ютера становить:

$$C_k = 4,25 + 0,26 + 1,18 = 5,69 \frac{\text{грн}}{\text{год}}.$$

Витрати комп'ютерного часу на створення програмного продукту становлять:

$$32 = 5,69 \cdot 258,8 = 1472,57 \text{ грн.}$$

Результати розрахунку амортизаційних відрахувань, витрат на електроенергію та собівартості комп'ютерного часу наведено на рисунку 4.3.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		48

Витрати на оплату праці розробника становлять:

$$31 = 72589,28 \text{ грн.}$$

Витрати комп'ютерного часу становлять:

$$32 = 1472,57 \text{ грн.}$$

Непрямі витрати приймаємо у розмірі 10% від суми витрат на оплату праці та комп'ютерного часу:

$$33 = (72589,28 + 1472,57) \cdot 0,10 = 7406,19 \text{ грн.}$$

Тоді витрати на створення програмного продукту становлять:

$$K3 = 72589,28 + 1472,57 + 7406,19 = 81468,04 \text{ грн.}$$

Витрати на обладнання становлять 29000 грн. Витрати на ліцензійні програмні продукти становлять 0 грн, оскільки для розробки використовуються безкоштовні інструменти та технології.

Загальна вартість створення програмного продукту становить:

$$K = 29000 + 0 + 81468,04 = 110468,04 \text{ грн.}$$

Результати підсумкового розрахунку собівартості програмного продукту наведено на рисунку 4.4.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		50

B56

✕

✓

fx

= (B54 + B55) * 10%

Book1.xlsx

	A	B	C
49			
50			
51	Показник	Значення, грн	
52	Витрати на обладнання, К1	29000	
53	Витрати на ліцензійні програмні продукти, К2	0	
54	Витрати на оплату праці, 31	72589.28	
55	Витрати комп'ютерного часу, 32	1472.57	
56	Непрямі витрати, 33	7406.185	
57	Витрати на створення ПП, К3	81468.035	
58	Загальна вартість створення ПП, К	110468.035	
59			

Рисунок 4.4 – Розрахунок собівартості програмного продукту

5 Охорона праці

Охорона праці є важливою складовою організації робочого процесу під час розробки програмного забезпечення. Основною метою охорони праці є створення безпечних, комфортних і здорових умов для працівника, запобігання професійним захворюванням, зменшення впливу шкідливих факторів та забезпечення правильного використання комп'ютерної техніки.

5.1 Оцінка умов роботи під час розробки програмного продукту

Розробка вебплатформи виконується в робочому приміщенні (рисунок 5.1), обладнаному комп'ютером, монітором, клавіатурою, мишею, робочим столом та кріслом. Основними видами робіт є написання програмного коду, проєктування бази даних, тестування функціоналу, налагодження серверної та клієнтської частини, а також підготовка пояснювальної записки.

Характеристики робочого приміщення наведено в таблиці 5.1

Таблиця 5.1 – Характеристика робочого приміщення

Назва приміщення	Висота, м	Довжина, м	Ширина, м	Площа, м ²	Об'єм, м ³	Орієнтація вікон
Робоча кімната	3	5	3	15	45	Північна

Площа приміщення становить 15 м², а об'єм – 45 м³. Такі показники є достатніми для організації одного робочого місця з персональним комп'ютером. У приміщенні перебачено природне освітлення через вікно, штучне освітлення, вентиляцію та достатній простір для розміщення обладнання.

Під час роботи за комп'ютером на працівника можуть впливати такі фактори: зорове напруження, статичне навантаження на м'язи спини та шиї, монотонність роботи, нервово-психічне навантаження, незначний шум від обладнання, електричним струмом у разі несправності обладнання.

Тривала робота за монітором може спричинити втому очей, сухість, почервоніння, головний біль та зниження концентрації. Постійне перебування в сидячому положенні може призвести до болю в спині, шиї, плечах і порушення кровообігу. Тому важливо правильно організувати робоче місце та режим праці й відпочінку.

Схема розміщення обладнання на робочому місці

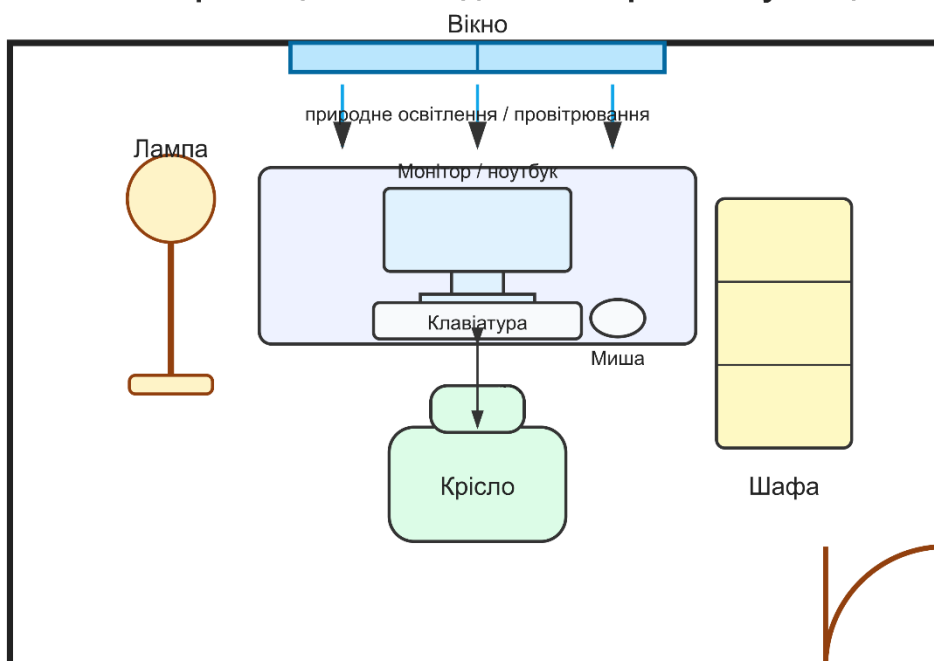


Рисунок 5.1 – Схема розміщення обладнання на робочому місці

5.2 Організація безпечного та ергономічного робочого місця

Для забезпечення безпечних умов праці робоче місце організовано відповідно до вимог ергономіки. Комп'ютерне обладнання розміщується таким чином, щоб користувач міг працювати без зайвого фізичного

напруження. Монітор встановлюється прямо перед користувачем, клавіатура та миша розташовується на зручній відстані, а робоче крісло забезпечує підтримку спини.

Під час організації робочого місця необхідно дотримуватися таких вимог:

- відстань від очей до екрана монітора повинна становити приблизно 500 – 700 мм;
- верхній край екрана має бути на рівні очей або трохи нижче;
- клавіатура та миша повинні розташовуватися так, щоб руки знаходилися у зручному положенні;
- робоче крісло повинно мати регулювання висоти та підтримку спини;
- простір для ніг має бути вільним і достатнім для комфортного положення;
- робоче місце необхідно підтримувати в чистоті та порядку.

Для зменшення втоми під час роботи за комп'ютером необхідно дотримуватися раціонального режиму праці та відпочинку. Рекомендується роботи короткі перерви після кожної години роботи, виконувати вправи для очей, змінювати положення тіла та періодично провітрювати приміщення.

Робота з комп'ютером повинна виконуватися лише за справного обладнання. Перед початком роботи необхідно перевірити стан кабелів, розеток, монітор, клавіатури та інших пристроїв. Забороняється працювати з пошкодженими проводами, несправними розетками або обладнанням, яке перегрівається.

5.3 Санітарно-гігієнічні умови праці користувача ПК

Санітарно-гігієнічні умови праці мають важливе значення для збереження здоров'я користувача комп'ютера. До основних показників

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		54

належать мікроклімаць приміщення, освітлення, рівень шуму, вентиляція та електромагнітне випромінювання.

Основні санітарно-гігієнічні показники робочого приміщення наведено в таблиці 5.2.

Таблиця 5.2 – Санітарно-гігієнічні показники робочого приміщення

Фактор	Нормоване значення	Фактичне значення
Температура повітря	18-25 °С	20-22 °С
Вологість повітря	40-60%	45-55%
Швидкість руху повітря	0,1-0,2 м/с	0,1-0,2 м/с
Освітленість робочої поверхні	300-500 лк	350-450 лк
Рівень шуму	До 40 дБА	До 35 дБА
Тип вентиляції	Природна або штучна	Природна
Режим роботи	З перервами	З перервами

У приміщенні забезпечується природна вентиляція шляхом регулярного провітрювання. Температура повітря підтримується на комфортному рівні, що дозволяє уникнути перегрівання або переохолодження працівника і сприяє комфортній роботі за комп'ютером.

Освітлення робочого місця є змішаним: використовується природне освітлення через вікно та штучне освітлення за допомогою світлодіодної лампи. Робочий стіл освітлюється рівномірно, без різки тіней і відблисків на екрані монітора. Це дозволяє зменшити навантаження на органи зору.

Рівень шуму в приміщенні є низьким. Основними джерелами шуму можуть бути вентилятори комп'ютера, клавіатура та інші периферійні пристрої. Оскільки рівень шуму не перевищує допустимих значень, він не має значного негативного впливу на працездатність

5.4 Вимоги електробезпеки під час роботи з комп'ютерною технікою

Під час роботи з комп'ютерною технікою необхідно дотримуватися правил електробезпеки та експлуатації обладнання. Робоче приміщення належить до приміщень без підвищеної небезпеки, оскільки в ньому відсутня підвищена вологість, струмопровідний пил, агресивне середовище та інші фактори, які можуть збільшити ризик ураження електричним струмом.

Електроживлення комп'ютерного обладнання здійснюється від мережі 220 В, 50 Гц. Для підвищення безпеки використовуються справні розетки, мережевий фільтр та заземлення. Кабелі живлення розміщуються так, щоб вони не заважали пересуванню та не створювали ризику механічного пошкодження.

Для забезпечення безпечної роботи необхідно дотримуватися таких правил:

- перед початком роботи перевіряти справність обладнання;
- не використовувати пошкоджені кабелі, вилки або розетки;
- не торкатися електрообладнання мокрими руками;
- не розміщувати рідину біля комп'ютера або ноутбука;
- не закривати вентиляційні отвори обладнання;
- після завершення роботи правильно вимикати комп'ютер і периферійні пристрої.

Особливу увагу потрібно приділяти запобіганню перегріву обладнання. Для цього комп'ютер або ноутбук повинен розташовуватися на рівній поверхні з доступом повітря до вентиляційних отворів. Регулярне очищення робочого місця від пилу також знижує ризик перегріву та несправностей.

5.5 Пожежна безпека робочого приміщення

Під час роботи з комп'ютерною технікою існує ризик виникнення пожежі через коротке замикання, перевантаження електромережі, несправність обладнання або неправильну експлуатацію електроприладів. У робочому приміщенні можуть знаходитися горючі матеріали: меблі, папір, штори, ізоляція проводів, корпуси периферійних пристроїв.

Для запобігання пожежі необхідно дотримуватися організаційних і технічних заходів протипожежної безпеки. До організаційних заходів належать підтримання порядку на робочому місці, заборона використання несправного обладнання, своєчасне вимкнення електроприладів після завершення роботи та ознайомлення користувача з правилами пожежної безпеки.

До технічних заходів належать використання справної електропроводки, автоматичних вимикачів, мереживих фільтрів, заземлених розеток і недопущення перевантаження електромережі. Не допускається підключення великої кількості пристроїв до одного подовжувача.

У приміщенні повинен бути забезпечений вільний доступ до виходу та електрощитової. Проходи не повинні захаращуватися сторонніми предметами. У разі виникнення пожежі необхідно негайно вимкнути електроживлення, повідомити відповідні служби та використати первинні засоби пожежогасіння.

Для гасіння електрообладнання рекомендується використати вуглекислий вогнегасник, оскільки він не пошкоджує електроніку та може застосовуватися для обладнання, що перебуває під напругою. Вогнегасник повинен бути розміщений у доступному місці.

5.6 Профілактика перевантажень під час роботи за комп'ютером

Під час розробки вебплатформи основним робочим інструментом є персональний комп'ютер або ноутбук. Тому важливо забезпечити правильну організацію робочого місця, дотримання режиму праці та зниження впливу шкідливих факторів на користувача.

Для зменшення навантаження на зір необхідно налаштувати яскравість і контрастність екрана відповідно до освітлення приміщення. Монітор потрібно розташовувати так, щоб на нього не потрапляли прямі сонячні промені та не виникали відблиски.

Для профілактики порушень опорно-рухового апарату необхідно правильно сидіти за робочим місцем. Спина повинна мати опору, плечі мають бути розслабленими, а руки повинні вільно лежати на столі або підлокітниках. Ноги мають стояти на підлозі або на спеціальній підставці. Під час перерв бажано виконувати легку розминку для ший, спини, рук і плечей.

З метою зменшення нервово-психічного навантаження роботу необхідно планувати за етапами. Доцільно чергувати написання програмного коду, тестування, роботу з документацією та короткі перерви.

Висновки

У результаті виконання кваліфікаційної роботи було створено вебплатформу для створення та проведення тестувань. Система призначена для використання в освітньому процесі та дозволяє викладачам створювати тести, додавати питання й відповіді, а студентам – проходити тестування та отримувати результати.

Під час розробки було використано React для клієнтської частини, Spring Boot для серверної логіки та MySQL для зберігання даних. У платформі реалізовано авторизацію користувачів, розподіл ролей на адміністратора, викладача і студента, автоматичне оцінювання результатів, приватні тести з кодом доступу та можливість додавання зображень до питань.

У роботі також було виконано економічні розрахунки вартості створення програмного продукту та розглянуто питання охорони праці під час роботи з комп'ютерною технікою

Отже, поставлену мету кваліфікаційної роботи було досягнуто. Розроблена вебплатформа є функціональним програмним продуктом, який може використовуватися для організації контролю знань у навчальних закладах і має можливість подальшого розширення.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		59

Список використаних джерел

1. React. Quick Start – React. URL: <https://react.dev/learn> (дата звернення: 21.05.2026).
2. Spring. Guides. URL: <https://spring.io/guides> (дата звернення: 21.05.2026).
3. MySQL. MySQL Documentation. URL: <https://dev.mysql.com/doc/> (дата звернення: 21.05.2026).
4. IntelliJ IDEA. IntelliJ IDEA Help. URL: <https://www.jetbrains.com/help/idea/getting-started.html> (дата звернення: 23.05.2026).
5. Visual Studio Code. Learn Visual Studio Code. URL: <https://code.visualstudio.com/learn> (дата звернення: 23.05.2026).
6. Sass: Syntactically Awesome Style Sheets. Sass: Documentation. URL: <https://sass-lang.com/documentation/> (дата звернення: 25.05.2026).
7. Axios. First steps. URL: <https://axios.rest/pages/getting-started/first-steps> (дата звернення: 25.05.2026).
8. React Router. React Router Official Documentation. URL: <https://reactrouter.com/home> (дата звернення: 26.05.2026).
9. Webpack. Documentation. URL: <https://webpack.js.org/concepts/> (дата звернення: 27.05.2026).
10. npm Docs. URL: <https://docs.npmjs.com/> (дата звернення: 27.05.2026).
11. Node.js. Node.js v26.3.0 documentation. URL: <https://nodejs.org/api/all.html> (дата звернення: 27.05.2026).
12. Apache Maven Project. Documentation. URL: <https://maven.apache.org/guides/> (дата звернення: 28.05.2026).
13. Krzysztof Sacha. Inżynieria Oprogramowania. Warszawa. Wydawnictwo Naukowe PWN. 2010. 417с.

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		60

Додаток А

Лістинг А.1 – Фрагмент програмного коду налаштування безпеки та розмежування доступу користувачів

```
@Configuration
@RequiredArgsConstructor
public final SecurityConfig {

    private final JwtFilter jwtFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception{
        http.cors(cors -> {}).csrf(csrf ->
        csrf.disable()).authorizeHttpRequests(auth ->
        auth.requestMatchers("/auth/**").permitAll().requestMatchers("/t
        ests/teacher/**").hasAnyRole("TEACHER",
        "ADMIN").requestMatchers("/tests").authenticated().requestMatche
        rs("/questions/test/**").authenticated().requestMatchers("/quest
        ions/**").hasAnyRole("TEACHER",
        "ADMIN").requestMatchers("/answers/**").hasAnyRole("TEACHER",
        "ADMIN").requestMatchers("/results/submit").authenticated().requ
        estMatchers("/results/student/**").authenticated().requestMatche
        rs("/users/**").hasRole("ADMIN").requestMatchers(HttpMethod.GET,
        "/users/**").hasRole("ADMIN").requestMatchers(HttpMethod.POST,
        "/users/**").hasRole("ADMIN").requestMatchers(HttpMethod.PUT,
        "/users/**").hasRole("ADMIN").requestMatchers(HttpMethod.DELETE,
        "/users/**").hasRole("ADMIN").requestMatchers("/uploads/**").per
        mitAll().anyRequest().authenticated()).addFilterBefore(jwtFilter
        , UsernamePasswordAuthenticationFilter.class);
        return http.build();

    @Bean
    public PasswordEncoder passwordEncoder(){
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager
    authenticationManager(AuthenticationConfiguration config) throws
    Exception{
        return config.getAuthenticationManager();
    }
}
```

Додаток Б

Лістинг Б.1 – Фрагмент програмного коду реєстрації та авторизації користувачів

```
@Service
@RequiredArgsConstructor
public class AuthService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;

    public String register(RegisterRequest request) {
        if (userRepository.existsByEmail(request.getEmail())) {
            throw new RuntimeException("Email already exists");
        }
        User user =
        User.builder().email(request.getEmail()).password(passwordEncoder
        r.encode(request.getPassword())).role(Role.STUDENT).build();
        userRepository.save(user);
        return jwtService.save(user);
    }

    public String login(LoginRequest request) {
        User user =
        userRepository.findByEmail(request.getEmail()).orElseThrow(() ->
        new RuntimeException("User not found");
        if (!passwordEncoder.matches(request.getPassword(),
        user.getPassword())) {
            throw new RuntimeException("Wrong password");
        }
        return jwtService.generateToken(user);
    }
}
```

Додаток В

Лістинг В.1 – Фрагмент програмного коду налаштування доступу до завантажених зображень

```
@Configuration
public class StaticConfig implements WebMvcConfigurer{
    @Override
    public void addResourceHandlers(ResourceHandlerRegistry
registry){
        registry.addResourceHandler("/uploads/**").addResourceLocations("file:uploads/");
    }
}
```

					КР ФМБ 2026 123.3 013 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		63

Додаток Г

Показник	Формула для розрахунку
1. Капіталовкладення у створення ПП	$K = K_1 + K_2 + K_3$, де K_1 – витрати на обладнання, грн.; K_2 – витрати на ліцензійні програмні продукти, грн.; K_3 – витрати на створення ПП, грн.
2. Витрати програмістів-розробників праці	$Z_1 = \sum_{k=1}^k P_k \times ZP_{\text{год}}^k + ZП\text{дод} \times T_k \times K_{\text{зп}}$, де P_k – кількість розробників k-й професії, чол.; $ZP_{\text{год}}^k$ – годинна зарплата розробника k-й професії, грн.; $ZП\text{дод}$ – додаткова заробітна плата k-й професії, грн.; T_k – трудомісткість розробки для k-го розробника, год. $K_{\text{зп}}$ – коефіцієнт нарахувань на фонд заробітної плати, частки;
3. Трудомісткість	$t = tH + ta + tb + tn + td + tv$, де t – трудомісткість створення програмного продукту, год.; tH – витрати на підготовку опису завдання, год.; ta – витрати на розробку алгоритму, год.; tb – витрати на створення бази даних, год.; tn – витрати на створення системи управління життєвим циклом програмного забезпечення на основі готової структури, год.; td – витрати праці на підготовку документації, год.; tv – витрати на відлагодження продукту при комплексному відлагодженні завдання, год.
4. Витрати комп'ютерного часу	$Z_2 = S_k \times T_{\text{пр}}$, де S_k – собівартість комп'ютерного години, грн.; $T_{\text{пр}}$ – витрати комп'ютерного часу на розробку програми, час.
5. Собівартість комп'ютерного години	$S_k = CA + CE + CTO$, де CA – амортизаційні відрахування, грн.; CE – енерговитрати, грн.; CTO – витрати на техобслуговування, грн.
6. Амортизаційні відрахування	$A = (В\text{поч.} - В\text{лік.}) / (T_{\text{к.в.}} \times 12)$, де $В\text{поч.}$ – балансова вартість обладнання, грн., $В\text{лік.}$ – ліквідаційна вартість обладнання, грн., $T_{\text{к.в.}}$ – термін корисного використання обладнання, цілі місяці. t – кількість місяців експлуатації обладнання.
7. Енерговитрати	$С_e = P_e \times S_{\text{кВт}}$ грн., де P_e – витрата електроенергії, споживаної комп'ютером за год; $S_{\text{кВт}}$ – вартість 1 кВт / год електроенергії, грн.;
8. Витрати на техобслуговування	$C_{\text{то}} = ZP_{\text{год}}^{\text{обсл.}} \times \gamma$, де $ZP_{\text{год}}^{\text{обсл.}}$ – годинна зарплата працівника обслуговуючого обладнання, грн. γ – періодичність обслуговування.
9. Періодичність обслуговування	$\lambda = N_{\text{то}} \div F_{\text{міс}}$, де $N_{\text{то}}$ – кількість обслуговувань обладнання в місяць; $F_{\text{міс}}$ – місячний фонд часу роботи обладнання, год.
10. Непрямі витрати	$Z_3 = VO_{\text{прим.}} + C_1 + C_2$, де $VO_{\text{прим.}}$ – вартість оренди приміщення, грн; C_1 – витрати на утримання приміщень (на освітлення, опалення охорону і прибирання), грн. Складають 5-15% від вартості приміщень; C_2 – інші витрати, грн. Складають 10-20% від вартості капітальних вкладень.

Рисунок Г.1 – Формули для розрахунку трудомісткості та собівартості програмного продукту